

STUDY GUIDE · GIT & GITHUB · FROM BEGINNER TO ADVANCED

Git Practical Course

Git & GitHub From Beginner to Advanced

Every one of the 98 lessons on a page or less: what it teaches, its diagram, its commands and its recap. Then the DevOps workflow from branch to Kubernetes, the troubleshooting method, the command reference, the capstone, a glossary and 35 interview questions.

Sufyan Ahmad · DevOps Engineer

<https://github.com/sufyanahmadkamboh/sufyan-devops-git-practical-course>

Contents

Study material

The method and the big picture

Module 01 · Git fundamentals

Module 02 · Repositories

Module 03 · Commits

Module 04 · Git history

Module 05 · Branches

Module 06 · Merging

Module 07 · Undoing changes

Module 08 · Stash

Module 09 · Remote repositories

Module 10 · GitHub fundamentals

Module 11 · Pull requests

Module 12 · Collaboration

Module 13 · Rebase

Module 14 · Advanced Git

Module 15 · Git internals

Module 16 · Advanced recovery

Module 17 · Git hooks

Module 18 · Advanced repositories

Module 19 · Git security

Module 20 · GitHub advanced

Module 21 · Professional DevOps Git workflow

Module 22 · Real-world troubleshooting

Git command reference lab

Module 24 · Final capstone

Glossary

Interview questions

The full lessons, with every command and its real, tested output, the exercises, challenges, assessments, troubleshooting labs and projects, are in the repository.

Study material

Everything you need to revise after (or alongside) the hands-on lessons.

File	What it is	Use it when
<code>study-guide.pdf</code>	every lesson on a page or less (concept, diagram, commands, recap), the DevOps workflow, troubleshooting method, command reference, capstone, glossary and interview questions, as one printable PDF (built by <code>study/tools/build_pdf.py</code>)	you want to revise offline, print or annotate
<code>glossary.md</code>	every term used in the course in plain words, with the lesson that explains it	a word in a lesson is unclear
<code>interview-questions.md</code>	35 questions with model answers, from "Git vs GitHub" to supply-chain security and release pipelines	before an interview, or to test yourself
<code>../docs/command-reference.md</code>	every command by purpose, linked to its lesson	you know what you want to do, not the command

How to study

1. Take the lessons in order, typing every command. Do the **Break it** part every time: it is where most of the learning happens.
2. Finish each module with its `assessment.md`. Not sure about a quiz answer? Re-run the lesson's lab and look.
3. After modules 09, 16 and 22, do a **project** without looking at the reference solution.
4. Revise with the study guide: for each lesson, explain the recap out loud, then check the full lesson if you could not.
5. End with the **capstone** and the **final exam**.

A suggested pace: one module per study session (1–2 hours), about five weeks part-time for the whole course.

Rebuild the PDF

```
pip install markdown
python study/tools/build_pdf.py (uses a headless Chrome or Edge to print the PDF)
```

The method and the big picture

How every lesson works

Each concept gets its own lesson, and every lesson follows the same path:

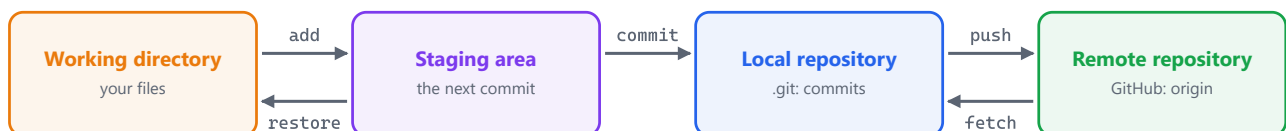
Understand → Visualize → Execute → Observe → Break → Troubleshoot → Fix → Practice → Challenge → Real world

Section	What you do
What are we learning?	the concept in plain words
Visual	a diagram of what Git does
Lab setup	one command creates a practice repository with a known history (same commit IDs as the lesson)
Demonstration	real commands, real outputs, explained
Command breakdown	the commands and options, as a table
Hands-on exercise	instructions → expected result → verification command
Break it → Troubleshoot → Fix	a realistic mistake on purpose, how to read the error, how to fix it
Real-world example	how the concept shows up in a DevOps team
Practice challenge	a harder task, solution hidden until you open it
Recap	three bullet points

Each lesson folder also contains `commands.md`, `exercise.md`, `challenge.md` and `troubleshooting.md`, generated from the lesson, and every module ends with an `assessment.md`: a quiz, a practical challenge, a troubleshooting challenge on a broken repository, and a real-world scenario.

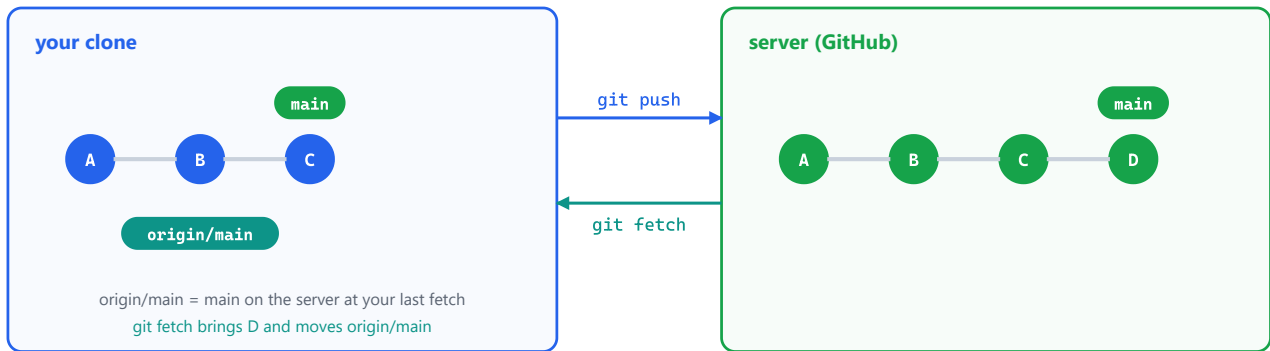
Diagrams

Where your changes live



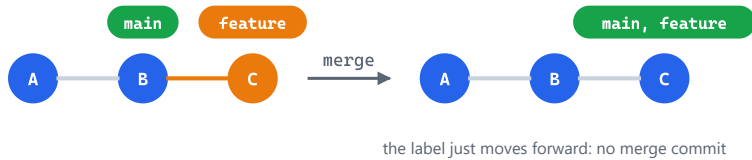
git status shows the first two columns · git log shows the third · git fetch updates origin/*
git pull = fetch + merge (or rebase) · git switch moves the working directory to another commit

Local branches, remote-tracking branches and the server

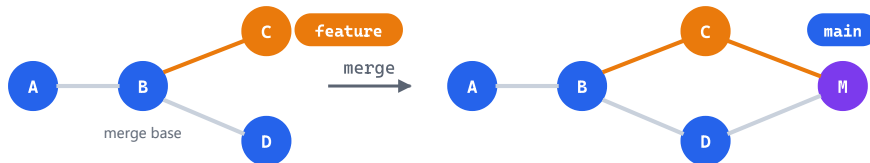


Branches and the two kinds of merge

fast-forward: main had not moved

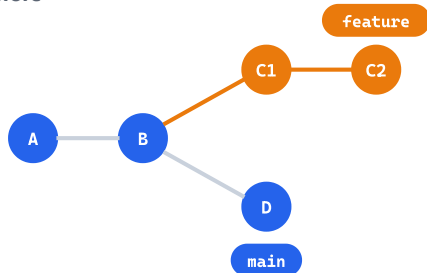


three-way: both branches moved



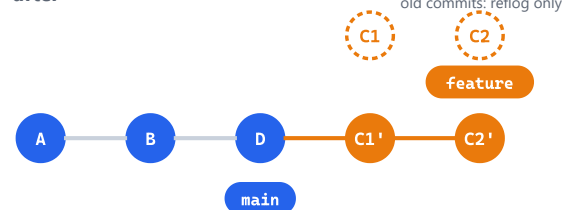
Rebase: replay your commits on a new base

before



`git rebase main`

after



C1' and C2' are new commits (new IDs). Never rebase commits others have built on.

git reset: what each mode resets

	HEAD / branch	Staging area (index)	Working directory
--soft	moves	kept	kept
--mixed (default)	moves	reset	kept
--hard	moves	reset	reset (uncommitted work lost!)

Commits left behind by reset are still recoverable with git reflog / ORIG_HEAD (lessons 33, 81).

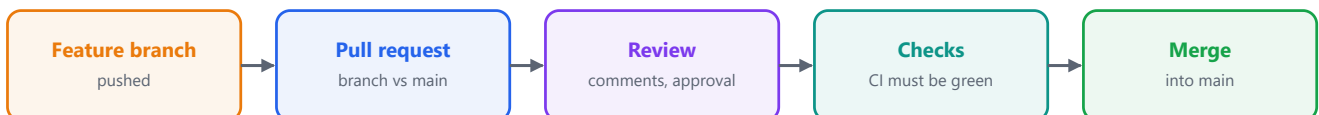
The reflog: where HEAD has been

```
HEAD@{0}    reset: moving to HEAD~3
HEAD@{1}    commit: Price mocha
HEAD@{2}    commit: Add mocha
HEAD@{3}    commit: Price green tea
HEAD@{4}    checkout: moving from main to v1.0.0
```

→ `git reset --hard HEAD@{1}`
or `git branch rescue HEAD@{1}`

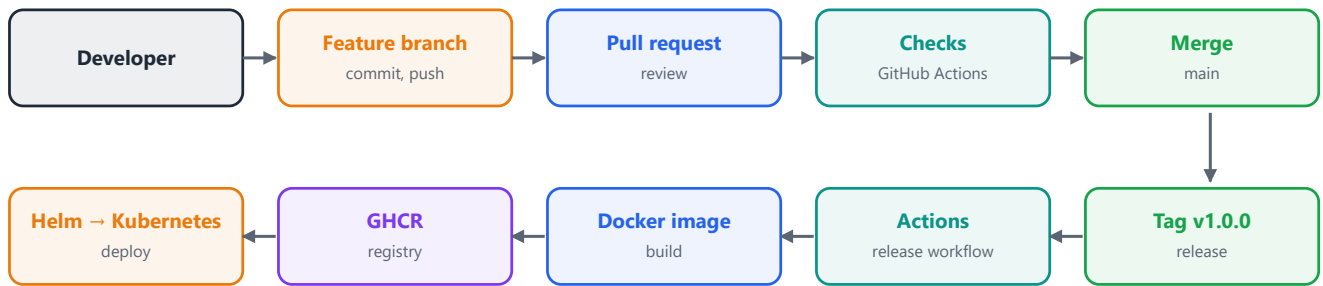
local to your clone, never pushed;
entries expire after 30–90 days

A pull request



Protected main: no direct pushes, required review and checks (lesson 59)

From a developer's commit to Kubernetes



Module 21 and the capstone run this whole chain; rollback = redeploy the previous tag

Module 01 · Git fundamentals

Lesson 01 · What is Git?

Why version control exists, by first working **without** it, then doing the same work with Git and comparing.

VISUAL

Without Git	With Git
project-final	Commit 1 "Add the menu"
project-final2	↓
project-final-final	Commit 2 "Add prices"
project-final-final-v2	↓
project-final-real ← which one?	Commit 3 "Raise the latte price" ← who, when, why: recorded

Git is a **version control system**: it records snapshots of your project (commits), who made each one, when, and why, and it can show the difference between any two of them or bring any of them back.

COMMANDS

Command	What it does here
<code>git init -q -b main</code>	turn the current folder into a Git repository, first branch <code>main</code> (<code>-q</code> : quietly)
<code>git config user.name/email</code>	who you are, written into every commit (lesson 04 does this properly)
<code>git add prices.txt</code>	choose what goes into the next snapshot
<code>git commit -m "..."</code>	record the snapshot with a message
<code>git log --oneline</code>	the history, one line per commit
<code>git diff HEAD~1 HEAD</code>	what changed between the previous commit and the latest
<code>git show HEAD~2:prices.txt</code>	the file as it was two commits ago

RECAP

- Git records **snapshots** (commits) with author, date and message.
- Any version can be compared with any other, and brought back.
- Copies named `final-final` cannot tell you what changed, who changed it, or why.

Full lesson with every command and its real output: [01-what-is-git](#)

Lesson 02 · Git vs GitHub

The difference between **Git** (a program on your computer) and **GitHub** (a website that hosts Git repositories and adds collaboration). We prove that Git works with no GitHub at all, then talk to GitHub with plain Git.

VISUAL



GitHub is one host among several (GitLab, Bitbucket, Gitea, Azure Repos, a plain server with SSH). They all store the same kind of repository; Git is what you use with all of them.

COMMANDS

Command	What it does here
<code>git remote -v</code>	list the remote repositories this one knows (none at first)
<code>git init --bare</code>	create a repository without a working folder: what a server stores
<code>git remote add origin PATH</code>	register a remote named <code>origin</code> (a path, or an <code>https:// / git@</code> URL)
<code>git push origin main</code>	send the <code>main</code> branch's commits to the remote
<code>git ls-remote --heads URL</code>	ask a remote which branches it has, without cloning

RECAP

- **Git** is the version control program; it works on your computer, offline, with no account.
- **GitHub** hosts Git repositories and adds collaboration: pull requests, reviews, issues, Actions, releases.
- A remote is just "another copy of this repository somewhere": a folder, a server, or GitHub.

Full lesson with every command and its real output: [02-git-vs-github](#)

Lesson 03 · Installing Git

How to install Git on Linux, macOS and Windows, check which Git you are running, and where it keeps its own files.

VISUAL

```
you → git (the CLI) → reads config: system → global (~/.gitconfig) → local (.git/config)
                        ↳ runs helpers: git-<command> programs in its "exec path"
                        ↳ works on:    the repository you are in (.git)
```

COMMANDS

Command	What it does
<code>git --version</code>	the installed version
<code>command -v git</code>	which <code>git</code> program runs (the first one on your <code>PATH</code>)
<code>git --exec-path</code>	the folder with Git's helper programs
<code>git <command> -h</code>	short usage; <code>git help <command></code> opens the full manual

RECAP

- Install Git with your package manager or the official installer; check with `git --version` .
- `git <command>` maps to built-ins and `git-<command>` programs: that is how extensions plug in.
- Typos produce "is not a git command"; `-h` and `git help` are always available offline.

Full lesson with every command and its real output: [03-installing-git](#)

Lesson 04 · First Git configuration

How to tell Git who you are, and how Git combines settings from three levels: system, global and local.

VISUAL

```
system  /etc/gitconfig (or the installer's)    every user on this machine
  ▼ overridden by
global  ~/.gitconfig                          you, in every repository
  ▼ overridden by
local   REPO/.git/config                      this repository only      ← the most specific wins
```

Your name and e-mail are written into **every commit** you make. They are not a login: GitHub matches the e-mail to an account to show your avatar, nothing more.

COMMANDS

Command	What it does
<code>git config --global KEY VALUE</code>	set a value in <code>~/.gitconfig</code>
<code>git config KEY VALUE</code>	set it in the current repository (<code>.git/config</code>)
<code>git config --list --show-origin --show-scope</code>	every effective setting, with its file and level
<code>git config --get KEY</code>	the value that wins
<code>git config --unset KEY</code>	remove a setting from one level

RECAP

- `user.name` and `user.email` go into every commit; set them before your first commit.
- Three levels: system < global < local. The most specific wins; `--show-origin --show-scope` tells you which.
- `init.defaultBranch main` names the first branch of new repositories.

Full lesson with every command and its real output: [04-first-configuration](#)

Module 02 · Repositories

Lesson 05 · What is a Git repository?

What makes a folder a Git repository: the hidden `.git` folder, and what it contains.

VISUAL

```
cafe/                                ← the WORKING DIRECTORY: your files, as you edit them
├─ README.md
├─ menu.txt
├─ prices.txt
└─ .git/                             ← the REPOSITORY: everything Git knows
    ├─ HEAD                         which branch you are on
    ├─ config                       this repository's settings (lesson 04)
    ├─ objects/                     every version of every file, every commit (compressed)
    ├─ refs/heads/                  branches: one small file per branch, holding a commit ID
    └─ index                         the staging area (lesson 10)
```

Delete `.git` and the folder is an ordinary folder again: the files stay, the history is gone.

COMMANDS

Command / file	What it shows
<code>ls -A</code>	including hidden entries such as <code>.git</code>
<code>.git/HEAD</code>	the current branch
<code>.git/refs/heads/<branch></code>	the commit a branch points to
<code>.git/objects/</code>	the content database: files, folders, commits
<code>git count-objects -v</code>	how many objects, and their size
<code>git rev-parse --git-dir</code>	where the repository of the current folder is

RECAP

- A repository is the `.git` folder; the files around it are the working directory.
- `HEAD` names the current branch; a branch is a file holding a commit ID.
- `fatal: not a git repository` means no `.git` here or above: wrong folder, or never initialised.

Full lesson with every command and its real output: [05-what-is-a-repository](#)

Lesson 06 · git init

Turning a folder into a repository with `git init`, and exactly what that does (and does not do).

VISUAL

```
git-demo/      git init      git-demo/
└─ (files)      ──────────>  └─ (files)      ← untouched: still untracked
                              └─ .git/         ← new, empty history
                                      HEAD → refs/heads/main (a branch with no commits yet)
```

COMMANDS

Command	What it does
<code>git init</code>	create <code>.git</code> in the current folder (safe to run again: it reinitialises, it does not wipe history)
<code>git init NAME</code>	create the folder <code>NAME</code> and a repository in it
<code>git init -b main</code>	choose the first branch's name explicitly
<code>git init --bare</code>	a repository without a working directory, for servers (lesson 02)

RECAP

- `git init` creates `.git`; existing files stay untracked until you add them.
- A new branch has no commit, so `refs/heads/main` does not exist until the first commit.
- Check where you are before `git init`; `git rev-parse --show-toplevel` shows which repository you are in.

Full lesson with every command and its real output: [06-git-init](#)

Lesson 07 · The working directory

The working directory is where you edit files. Git compares it with the last commit and tells you what is new, modified or deleted. We create, change and delete files and watch how Git sees each one.

VISUAL

last commit (HEAD)		working directory		what Git reports
menu.txt (3 lines)	vs	menu.txt (4 lines)	→	modified
prices.txt	vs	(gone)	→	deleted
(not there)	vs	hours.txt	→	untracked (new, never committed)
README.md	=	README.md	→	(nothing: unchanged files are not listed)

COMMANDS

Command	What it does
<code>git status</code>	compare the working directory (and staging area) with the last commit
<code>git status --short</code>	the same, one line per file: <code>M</code> modified, <code>D</code> deleted, <code>??</code> untracked
<code>git diff FILE</code>	the line-by-line changes not yet staged

RECAP

- The working directory is your files as they are right now; Git compares them with the last commit.
- Three states for a changed file: modified, deleted, untracked.
- Nothing is recorded until you commit; `git restore` brings back the committed version of a file.

Full lesson with every command and its real output: [07-working-directory](#)

Lesson 08 · git status

How to read every part of `git status`: the branch line, the remote line, and the three sections of changes. It is the command you will run more than any other.

VISUAL

```
On branch main                                ← where you are (lesson 18)
Your branch is ahead of 'origin/main' by 1 commit. ← compared with the remote (lesson 43), when there is one

Changes to be committed:                      ← STAGED: goes into the next commit (lesson 09)
    modified:   menu.txt

Changes not staged for commit:                 ← MODIFIED in the working directory, not staged
    modified:   prices.txt

Untracked files:                               ← NEW files Git has never seen committed
    hours.txt
```

COMMANDS

Command	Use
<code>git status</code>	the full report, with hints for the next commands
<code>git status -s / --short</code>	compact, two columns
<code>git status -sb</code>	compact plus the branch and ahead/behind line
<code>git status --ignored</code>	also list ignored files (lesson 73)

RECAP

- `git status`: branch, ahead/behind, then staged / not staged / untracked.
- Only *Changes to be committed* goes into the next commit.
- `git status -sb`: two columns (staging area, working directory) plus the branch line.

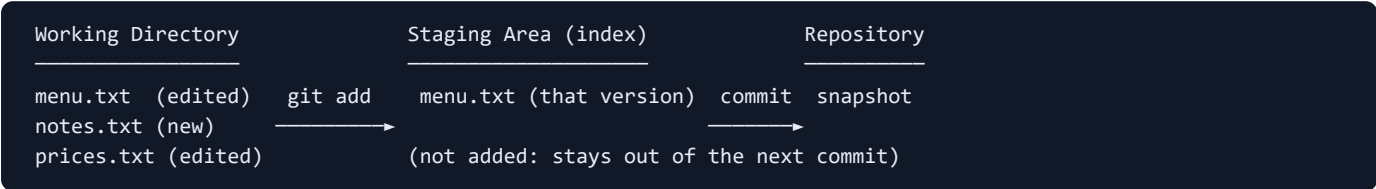
Full lesson with every command and its real output: [08-git-status](#)

Module 03 · Commits

Lesson 09 · git add

`git add` copies changes from the working directory into the **staging area**: the draft of your next commit. We compare `git add FILE` with `git add .` and `git add -A`.

VISUAL



COMMANDS

Command	Stages
<code>git add FILE...</code>	those files (or folders)
<code>git add .</code>	everything under the current folder
<code>git add -A</code>	everything in the repository
<code>git add -u</code>	only changes to files already tracked (no new files)
<code>git add -p</code>	interactively, hunk by hunk (pick parts of a file)
<code>git add -n .</code>	dry run: show what would be staged

RECAP

- `git add` copies the current content of a file into the staging area.
- `.` = this folder and below; `-A` = whole repository; `-u` = tracked files only; `-p` = hunk by hunk.
- Staged content is a snapshot: later edits need another `git add`.

Full lesson with every command and its real output: [09-git-add](#)

Lesson 10 · The staging area

Why Git has a staging area between your files and your commits, and how to see what is in it with `git diff` and `git diff --staged`.

VISUAL



The staging area lets you **build a commit from part of your changes**: two unrelated changes in your folder become two separate, clean commits.

COMMANDS

Command	Compares
<code>git diff</code>	working directory ↔ staging area (unstaged changes)
<code>git diff --staged</code> (= <code>--cached</code>)	staging area ↔ last commit (what the next commit contains)
<code>git diff HEAD</code>	working directory ↔ last commit (everything, staged or not)
<code>git diff --stat</code>	the same, summarised per file

RECAP

- The staging area is the draft of the next commit.
- `git diff` = not staged; `git diff --staged` = will be committed; `git diff HEAD` = everything.
- Stage deliberately to keep commits focused; check `git diff --staged` before committing.

Full lesson with every command and its real output: [10-staging-area](#)

Lesson 11 · git commit

How to record a commit, how to write a good commit message, and how to fix the last commit with `--amend`.

VISUAL

```
Working Directory —git add→ Staging Area —git commit→ new commit
                                     | snapshot of the staging area
                                     | author, date
                                     | message: WHY this change
                                     | parent: the previous commit
```

A good message:

```
Raise the latte price to 3.30      ← summary: imperative, ≤ 50 characters, no full stop
Milk prices went up 8 % this quarter. ← body (optional): why, not how; wrap at 72 characters
```

COMMANDS

Command	What it does
<code>git commit -m "summary"</code>	commit the staging area with that message
<code>git commit -m "summary" -m "body"</code>	summary + body paragraph
<code>git commit</code>	open the editor for the message
<code>git commit -a -m "..."</code>	stage modified tracked files first (not new ones)
<code>git commit --amend</code>	replace the last commit (new message and/or staged changes)
<code>git commit --amend --no-edit</code>	add staged changes to the last commit, keep the message

RECAP

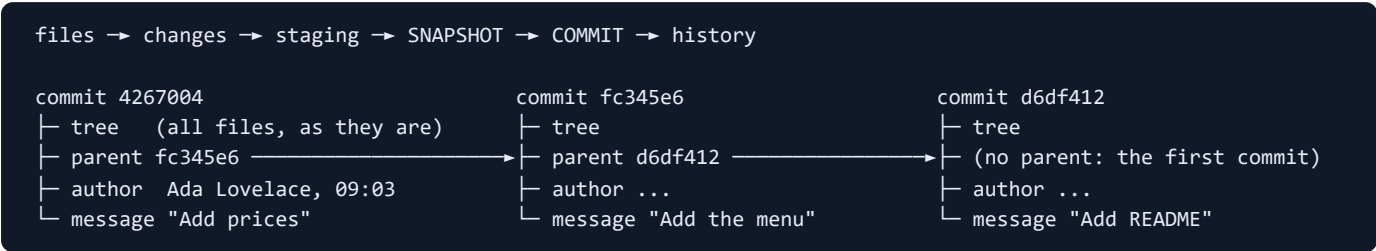
- `git commit` records the staging area as a snapshot with a message, author, date and parent.
- Message: short imperative summary, blank line, a body that explains why.
- `--amend` rewrites the last commit; only for commits nobody else has yet.

Full lesson with every command and its real output: [11-git-commit](#)

Lesson 12 · What actually happens during a commit?

What a commit is made of: a **snapshot** of all files (not a diff), plus an ID, an author, a timestamp and a link to its **parent** commit. Those parent links are the history.

VISUAL



The commit ID is a hash of all of that. Change anything (a file, the message, the parent) and the ID changes.

COMMANDS

Command	Shows
<code>git cat-file -p COMMIT</code>	the raw commit: tree, parents, author, committer, message
<code>git cat-file -p 'COMMIT^{tree}'</code>	the snapshot's top-level files and folders
<code>git rev-parse HEAD</code>	the full ID of the current commit
<code>git log --format='%H %P'</code>	each commit's ID and its parents' IDs

RECAP

- A commit = snapshot (tree) + parent(s) + author/committer + message; its ID is a hash of all of it.
- Git stores snapshots and reuses unchanged files; diffs are computed when you ask.
- Changing a commit changes its ID and the IDs of everything after it: history is a hash chain.

Full lesson with every command and its real output: [12-inside-a-commit](#)

Module 04 · Git history

Lesson 13 · git log

How to read the history: the full log, the one-line view, the graph of branches, and how to filter it by author, message, date and file.

VISUAL

```
git log                                     git log --oneline                       git log --oneline --graph --all
commit 4c3f... (HEAD -> main)              4c3f9a1 Price mocha                     * 4c3f9a1 (HEAD -> main) Price mocha
Author: Ada ...                          9b2e7c0 Add mocha                      | * 1d8e2f3 (feature-tea) Add green tea
Date: ...                                ...                                   | /
                                         * 4267004 Add prices
```

COMMANDS

Command	Shows
<code>git log</code>	every commit reachable from <code>HEAD</code> , full format
<code>git log --oneline</code>	short ID + summary
<code>git log --oneline --graph --all</code>	all branches, with the branch structure drawn
<code>git log -N</code>	only the last N commits
<code>git log -- FILE</code>	commits that changed FILE
<code>git log --grep=TEXT / --author=NAME / --since=DATE</code>	filter by message, author, date
<code>git log -p</code>	each commit with its diff
<code>git log --format='%h %an %ar %s'</code>	your own format (hash, author, relative date, subject)

RECAP

- `git log` shows the current branch's history, newest first; `--oneline`, `--graph`, `--all` change the view.
- Filter with `-- FILE`, `--grep`, `--author`, `--since`; compare branches with `A..B`.
- "Missing" commits are often on another branch: add `--all`.

Full lesson with every command and its real output: [13-git-log](#)

Lesson 14 · git show

How to inspect one commit: its metadata and its changes, a summary of the files it touched, and a file exactly as it was in that commit.

VISUAL

<code>git show COMMIT</code>	<code>git show --stat COMMIT</code>	<code>git show COMMIT:FILE</code>
commit, author, date message diff of every changed file	commit, author, date message prices.txt 1 +	the whole file, as it was in that commit

Ways to name a commit: a hash (`4267004`), a branch (`main`), `HEAD` , `HEAD~1` (one before), `HEAD~3` , a tag (`v1.0`).

COMMANDS

Command	Shows
<code>git show</code>	the latest commit (= <code>git show HEAD</code>) with its diff
<code>git show HASH / HEAD~N / BRANCH</code>	that commit
<code>git show --stat COMMIT</code>	files changed, insertions/deletions, no diff
<code>git show --name-only COMMIT</code>	only the file names
<code>git show COMMIT:PATH</code>	a file's content in that commit
<code>git show -s --format=... COMMIT</code>	metadata only, in your format

RECAP

- `git show COMMIT` = metadata + diff of one commit; `--stat` and `--name-only` summarise.
- `COMMIT:PATH` prints a file as it was in that commit.
- Commits can be named by hash, branch, tag, `HEAD` , `HEAD~N` .

Full lesson with every command and its real output: [14-git-show](#)

Lesson 15 · git diff

How to compare any two versions: your edits, what is staged, two commits, two branches; and how to read the diff format itself.

VISUAL

```
diff --git a/prices.txt b/prices.txt      ← which file
index 5813ff5..ddaed76 100644             ← blob IDs before..after, file mode
--- a/prices.txt                          ← "a" = the old side
+++ b/prices.txt                          ← "b" = the new side
@@ -1,3 +1,3 @@                           ← hunk: old lines 1-3, new lines 1-3
 espresso 2.50                            ← unchanged context line (starts with a space)
-latte 3.20                               ← removed
+latte 3.30                               ← added
 cappuccino 3.40
```

COMMANDS

Command	Compares
<code>git diff</code>	working directory ↔ staging area
<code>git diff --staged</code>	staging area ↔ HEAD
<code>git diff HEAD</code>	working directory ↔ HEAD
<code>git diff A B</code>	commit/branch A ↔ commit/branch B
<code>git diff A...B</code>	B ↔ the merge base of A and B (the branch's own changes)
<code>git diff --stat / --name-only</code>	summary only
<code>git diff --word-diff</code>	changed words instead of whole lines
<code>git diff -- FILE</code>	limit to a file

RECAP

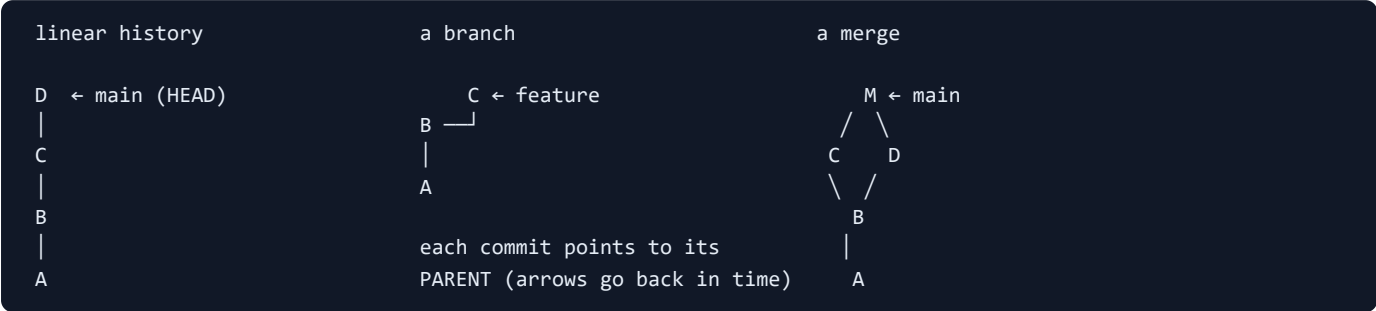
- `git diff` (unstaged), `--staged` (to be committed), `HEAD` (both), `A B` (two commits), `A...B` (a branch's own changes).
- Lines starting with `-` were removed, `+` added, space = unchanged context; `@@` marks a hunk.
- `--stat`, `--name-only`, `--word-diff` and `-- FILE` shape the output.

Full lesson with every command and its real output: [15-git-diff](#)

Lesson 16 · Commit history visualization

How to see history as what it really is: a graph of commits pointing to their parents, with branches as labels on some of them. We draw it, then build one and compare.

VISUAL



A commit has one parent (normal), two parents (a merge), or none (the first commit).

COMMANDS

Command	What it draws
<code>git log --oneline --graph --all</code>	the whole graph, every branch
<code>git log --graph --decorate --format='%h %d %s'</code>	with branch and tag labels (%d)
<code>git log --format='%h %p'</code>	each commit and its parents
<code>git log --first-parent</code>	only the main line, skipping the merged-in commits
<code>git show-branch</code>	a compact branch comparison

RECAP

- History is a graph: commits point to their parents; branches are labels on commits.
- `git log --oneline --graph --all` draws the whole graph; `--first-parent` follows the main line.
- A merge commit has two parents; the first commit has none.

Full lesson with every command and its real output: [16-history-visualization](#)

Module 05 · Branches

Lesson 17 · Why branches exist

The problem branches solve: several pieces of work in progress at once, without stepping on each other or on the stable version. We feel the problem first, on a single branch.

VISUAL

```
one branch for everything          a branch per piece of work

main ← Developer A (half-done login)    main (always releasable)
  ← Developer B (half-done payments)    |— feature-login      ← A works here
  ← Developer C (urgent bug fix)         |— feature-payments ← B works here
→ nothing on main is ever releasable    |— bugfix-prices    ← C fixes, merges, ships
```

COMMANDS

Command	What it does
<code>git switch -c NAME</code>	create a branch here and switch to it (lesson 20)
<code>git switch NAME</code>	switch to an existing branch (lesson 19)
<code>git log --oneline --graph --all</code>	see every branch at once

RECAP

- One branch for everything means nothing is ever releasable on its own.
- A branch per piece of work keeps `main` stable and lets work happen in parallel.
- Branches are cheap: create them for every feature, fix and experiment.

Full lesson with every command and its real output: [17-why-branches](#)

Lesson 18 · Creating branches

`git branch` lists and creates branches. Creating one does **not** switch to it: a common surprise.

VISUAL

```
before                                git branch feature-login           (you are still on main)

A — B — C ← main (HEAD)             A — B — C ← main (HEAD)
                                     ↘ feature-login           a new label on the same commit
```

A branch is a movable label pointing to a commit. Creating one copies nothing: it writes one small file with a commit ID.

COMMANDS

Command	What it does
<code>git branch</code>	list local branches, <code>*</code> = current
<code>git branch -v</code>	with the latest commit of each
<code>git branch NAME</code>	create NAME at the current commit (no switch)
<code>git branch NAME COMMIT</code>	create NAME at another commit
<code>git branch -a</code>	include remote-tracking branches (lesson 40)
<code>git branch -m OLD NEW</code>	rename a branch

RECAP

- `git branch NAME` creates a label at the current commit; it does not switch.
- `git branch` lists, `-v` adds the latest commit, `-m` renames.
- A branch is a file with a commit ID: creating one is instant, whatever the project size.

Full lesson with every command and its real output: [18-creating-branches](#)

Lesson 19 · Switching branches

`git switch` moves `HEAD` to another branch and updates your files to match it. We also meet the older `git checkout`, and what Git does with uncommitted changes when you switch.

VISUAL

```
git switch feature-tea
```

```
A — B — C      ← main
```

```
      |  
      D
```

```
      ← feature-tea (HEAD)
```

```
HEAD moved; the working directory now shows commit D
```

`git checkout` did this before Git 2.23, and also restored files, created branches and more. `git switch` (branches) and `git restore` (files) split that job into two clear commands. Both still work; prefer the new ones.

COMMANDS

Command	What it does
<code>git switch NAME</code>	move to branch NAME
<code>git switch -</code>	back to the previous branch
<code>git branch --show-current</code>	print the current branch
<code>git checkout NAME</code>	the older equivalent
<code>git switch --discard-changes NAME</code>	switch and throw away local changes (careful)

RECAP

- `git switch NAME` moves `HEAD` and updates your files; `git switch -` goes back.
- Uncommitted changes come along when they don't conflict; otherwise Git refuses, to protect your work.
- `git checkout` still works; `git switch` / `git restore` are clearer.

Full lesson with every command and its real output: [19-switching-branches](#)

Lesson 20 · Create and switch in one command

`git switch -c NAME` creates a branch and switches to it: the command you will use to start almost every piece of work. And starting it from the right place.

VISUAL

```
git switch -c feature-login = git branch feature-login + git switch feature-login
```

```
A — B — C ← main
           ↘ feature-login (HEAD) ← new label on the current commit, and HEAD on it
```

COMMANDS

Command	What it does
<code>git switch -c NAME</code>	create NAME at the current commit and switch to it
<code>git switch -c NAME START</code>	create it at START (a branch, tag or commit)
<code>git switch -C NAME</code>	create, or reset an existing NAME to here (overwrites: careful)
<code>git checkout -b NAME</code>	the older equivalent

RECAP

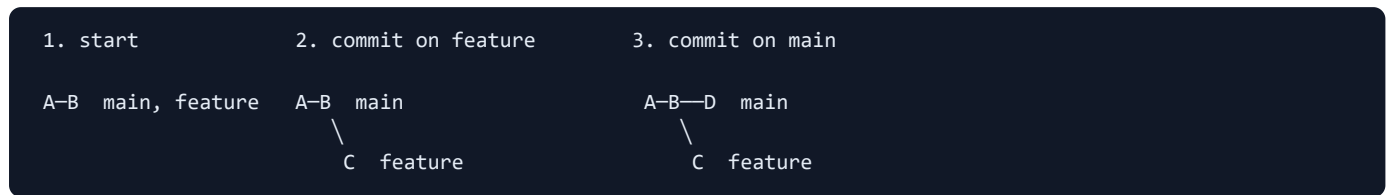
- `git switch -c NAME [START]` creates and switches in one step.
- A new branch contains everything of its starting point: start from the right place.
- `--onto` (lesson 61) repairs a branch started from the wrong place.

Full lesson with every command and its real output: [20-create-and-switch](#)

Lesson 21 · Branch visualization

Watching branches move: each commit moves the **current** branch's label forward, and only that one. We build a small history one commit at a time and draw it after every step.

VISUAL



COMMANDS

Command	Use
<code>git log --oneline --graph --all</code>	the drawing used in every step
<code>git log --graph --all --format='%h %d %s'</code>	the same with labels only where they are
<code>git rev-parse main feature</code>	the commit each label points to
<code>git merge-base main feature</code>	the commit where they forked (lesson 25)

RECAP

- A commit moves only the branch `HEAD` is on.
- Branches fork when two of them get different new commits; `git merge-base` finds the fork point.
- Draw the graph often; it answers most "where did my commit go" questions.

Full lesson with every command and its real output: [21-branch-visualization](#)

Lesson 22 · Deleting branches

How to delete branches safely with `-d`, why Git sometimes refuses, and when (and how carefully) to force it with `-D`.

VISUAL

```
git branch -d feature    merged into the current branch? → label removed, commits stay (they are on main)
                        NOT merged? → refused: its commits would be unreachable

git branch -D feature    removes the label anyway; unmerged commits become unreachable
                        (recoverable for a while with git reflog, lesson 79)
```

COMMANDS

Command	What it does
<code>git branch --merged / --no-merged</code>	branches whose work is / is not in the current branch
<code>git branch -d NAME</code>	delete if merged (safe)
<code>git branch -D NAME</code>	delete even if not merged (force)
<code>git push origin --delete NAME</code>	delete a branch on the remote (lesson 42)

RECAP

- `-d` deletes only merged branches; `-D` forces.
- Deleting a branch deletes a label; merged commits stay in history.
- Before `-D`, look at `git log main..BRANCH`: that is exactly what you are throwing away.

Full lesson with every command and its real output: [22-deleting-branches](#)

Module 06 · Merging

Lesson 23 · What is a merge?

Merging brings the work of one branch into another. We see the two shapes a merge can produce, and that merging **into** the current branch is always the direction.

VISUAL



`git merge X` means "bring X into the branch I am on". `HEAD`'s branch moves; `X` does not.

COMMANDS

Command	What it does
<code>git merge BRANCH</code>	bring BRANCH's commits into the current branch
<code>git merge --no-edit BRANCH</code>	accept the default merge message
<code>git merge -m "msg" BRANCH</code>	your own merge message
<code>git log --merges</code>	only merge commits

RECAP

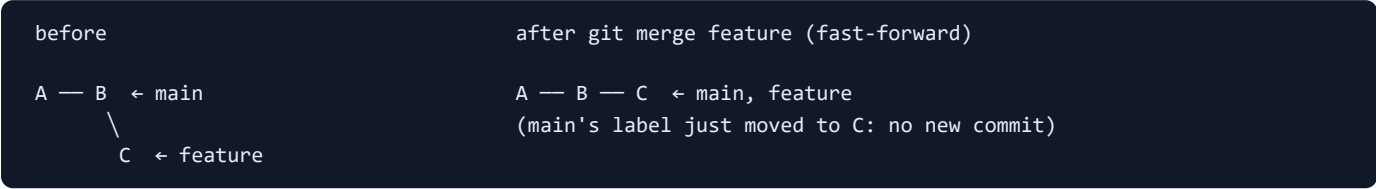
- `git merge X` brings X into the current branch; the current branch moves, X does not.
- A merge commit has two parents and combines both histories.
- Check `git branch --show-current` before merging: direction matters.

Full lesson with every command and its real output: [23-what-is-merge](#)

Lesson 24 · Fast-forward merge

When the current branch has not moved since the other branch started, a merge does not need a merge commit: Git simply moves the label forward. That is a fast-forward. We also see how to refuse it (`--no-ff`) and how to require it (`--ff-only`).

VISUAL



COMMANDS

Command	Behaviour
<code>git merge X</code>	fast-forward if possible, merge commit otherwise
<code>git merge --no-ff X</code>	always a merge commit
<code>git merge --ff-only X</code>	only fast-forward; refuse if a merge commit would be needed
<code>git config merge.ff only</code>	make <code>--ff-only</code> the default

RECAP

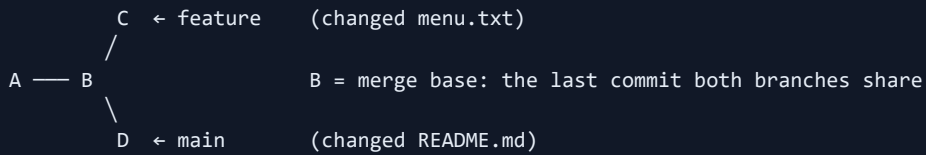
- Fast-forward: the current branch had not moved, so its label just moves forward. No merge commit.
- `--no-ff` forces a merge commit; `--ff-only` refuses anything but a fast-forward.
- Diverged branches cannot fast-forward: merge with a commit, or rebase first.

Full lesson with every command and its real output: [24-fast-forward](#)

Lesson 25 · Three-way merge

How Git merges two branches that both changed: it compares **three** snapshots, the two tips and their common ancestor, the **merge base**.

VISUAL



For every file: compare base→C and base→D.
changed on one side only → take that side
changed on neither → keep the base
changed on both sides → combine; if the same lines changed differently: CONFLICT (lesson 26)

COMMANDS

Command	Use
<code>git merge-base A B</code>	the common ancestor of A and B
<code>git diff BASE A</code>	what A changed since they split
<code>git diff A...B</code>	B's changes since the merge base (lesson 15)
<code>git merge B</code>	three-way merge into the current branch
<code>git show --stat MERGE</code>	what the merge brought in (compared with the first parent)

RECAP

- A three-way merge compares the two tips with their merge base.
- Changes on one side, or on different lines of the same file, merge automatically.
- No merge base = unrelated histories; `--allow-unrelated-histories` only when you really mean it.

Full lesson with every command and its real output: [25-three-way-merge](#)

Lesson 26 · Merge conflicts

What a merge conflict is, why it happens, and how to read the conflict markers Git writes into the file.

VISUAL

```
base:      latte 3.20
main:      latte 3.30  (Grace)
feature:   latte 3.50  (Ada)
           ↓ the same line changed differently on both sides: Git cannot choose

prices.txt after git merge:

espresso 2.50
<<<<<< HEAD          ← start: the version of the branch you are ON (main)
latte 3.30
=====              ← separator
latte 3.50
>>>>>> feature-tea   ← end: the version of the branch you are MERGING
cappuccino 3.40
```

COMMANDS

Marker / command	Meaning
<<<<<< HEAD	start of the current branch's version ("ours")
\ \ \ \ \ \ \ \ base	the common ancestor's version (with <code>diff3</code> / <code>zdiff3</code> style)
=====	separator
>>>>>> branch	end of the merged branch's version ("theirs")
<code>git checkout --conflict=diff3 FILE</code>	re-write the markers including the base
<code>git config merge.conflictStyle zdiff3</code>	always show the base (recommended)

RECAP

- A conflict = the same lines changed differently on both sides; Git stops and asks you.
- <<<<<< HEAD (yours) / ===== / >>>>>> branch (theirs); \|\|\|\|\|\|\|\| shows the base.
- You cannot commit until every conflict is resolved; `git status` lists them as "both modified".

Full lesson with every command and its real output: [26-merge-conflicts](#)

Lesson 27 · Resolving merge conflicts

The resolution loop: decide the right content, edit the file, `git add` it, commit the merge. And the shortcuts for "take mine" or "take theirs" for a whole file.

VISUAL

```
git merge X → CONFLICT → edit the file (remove ALL markers, keep the right content)
                        |
                        ▼
                    git add FILE      ← "this file is resolved"
                        |
                        ▼
                    git commit        ← records the merge (two parents)
```

COMMANDS

Command	Use
<code>git status</code>	which files are still "both modified"
edit, then <code>git add FILE</code>	mark a file resolved
<code>git commit</code>	conclude the merge
<code>git checkout --ours FILE / --theirs FILE</code>	take one side's whole file, then <code>git add</code> it
<code>git diff</code> during a merge	combined diff of what is still unresolved
<code>git mergetool</code>	open a configured visual merge tool

RECAP

- Resolve = write the correct content, remove every marker, `git add`, `git commit`.
- `--ours / --theirs` take one side's whole file.
- `git add` trusts you: check for leftover markers (`git diff --check`) and test before committing.

Full lesson with every command and its real output: [27-resolving-conflicts](#)

Lesson 28 · Aborting a merge

`git merge --abort` returns you to exactly where you were before a merge started. Useful when a conflict is bigger than expected, or when you merged the wrong branch.

VISUAL

```
git merge X → CONFLICT (half-merged files, markers, MERGE_HEAD)
                |
                └─ git merge --abort → back to the state before the merge: clean, as if nothing
                    happened
```

COMMANDS

Command	What it does
<code>git merge --abort</code>	cancel an in-progress merge, restore the pre-merge state
<code>git merge --quit</code>	forget the merge state but keep the files as they are now
<code>git status</code>	shows whether a merge is in progress

RECAP

- `git merge --abort` restores the state before the merge started.
- Start merges with a clean working directory (commit or stash first).
- `MERGE_HEAD` exists while a merge is in progress.

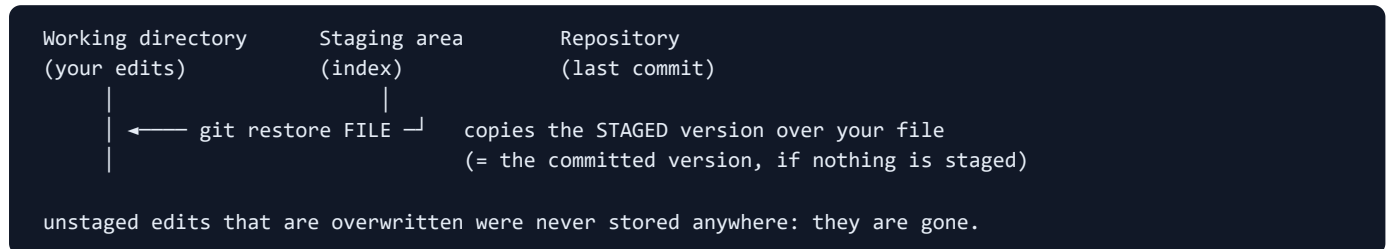
Full lesson with every command and its real output: [28-abort-merge](#)

Module 07 · Undoing changes

Lesson 29 · Undo working directory changes

How to throw away edits you have not staged yet: `git restore FILE`. And why this is one of the few Git commands that really destroys work.

VISUAL



COMMANDS

Command	What it does
<code>git restore FILE</code>	discard unstaged changes to FILE
<code>git restore .</code>	discard all unstaged changes below the current folder
<code>git restore -p FILE</code>	choose hunk by hunk what to discard
<code>git restore --source=COMMIT FILE</code>	take FILE as it was in COMMIT
<code>git checkout -- FILE</code>	the older command for the same thing
<code>git clean -n / -f</code>	new untracked files are not touched by restore: lesson 72

RECAP

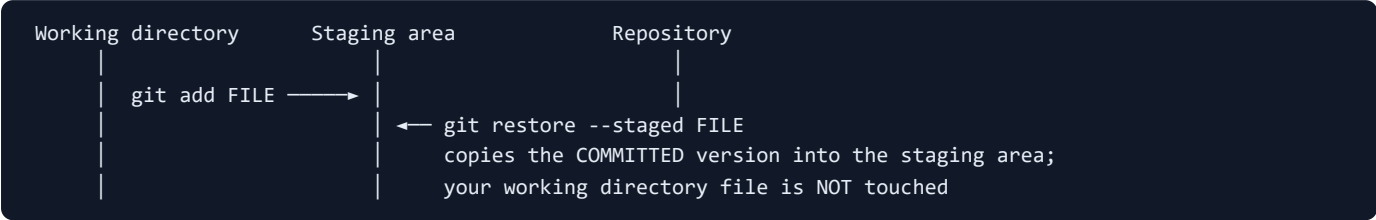
- `git restore FILE` discards unstaged edits; they cannot be recovered.
- It restores the **staged** version if there is one, otherwise the committed one.
- Read `git diff` before restoring; stage what you want to keep.

Full lesson with every command and its real output: [29-restore-working-directory](#)

Lesson 30 · Unstage files

How to take a file back out of the next commit without losing your edit: `git restore --staged FILE`. This undo is completely safe.

VISUAL



COMMANDS

Command	What it does
<code>git restore --staged FILE</code>	unstage FILE, keep the edit
<code>git restore --staged .</code>	unstage everything
<code>git restore --staged -p FILE</code>	unstage hunk by hunk
<code>git reset FILE</code>	the older command for the same thing (lesson 31)
<code>git rm --cached FILE</code>	stop tracking FILE but keep it on disk (lesson 73)

RECAP

- `git restore --staged FILE` removes a file from the next commit; the edit stays.
- It is always safe: it never touches your working directory.
- Before the first commit, use `git rm --cached FILE`.

Full lesson with every command and its real output: [30-unstage-files](#)

Lesson 31 · git reset: soft, mixed and hard

`git reset COMMIT` moves the current branch back to an earlier commit. Its three modes decide what happens to the work of the commits you step back over: keep it staged, keep it unstaged, or delete it.

VISUAL

before: A — B — C ← main (HEAD)

git reset ... B

		moves the branch	staging area	working directory	
--soft	B	yes	keeps C's work	keeps C's work	→ "uncommit", ready to recommit
--mixed	B	yes	reset to B	keeps C's work	→ "uncommit and unstage"
(default)					
--hard	B	yes	reset to B	reset to B	→ C's work is GONE from your folder

after: A — B ← main (HEAD)

C still exists, unreferenced: reflog can find it (lesson 33)

COMMANDS

Command	Branch	Staging area	Working directory
git reset --soft C	→ C	unchanged	unchanged
git reset C (--mixed)	→ C	= C	unchanged
git reset --hard C	→ C	= C	= C (uncommitted work lost!)
git reset FILE	—	FILE = HEAD	unchanged (= restore --staged)

`HEAD~1` is one commit back, `HEAD~3` three; `ORIG_HEAD` is where the branch was before the last reset.

RECAP

- `reset` moves the branch; `--soft` keeps changes staged, `--mixed` unstaged, `--hard` deletes them.
- Commits removed by reset are recoverable (reflog, `ORIG_HEAD`); uncommitted work removed by `--hard` is not.
- Never reset commits others already have: use `revert` .

Full lesson with every command and its real output: [31-git-reset](#)

Lesson 32 · git revert

`git revert COMMIT` creates a **new** commit that undoes an earlier one. History is not rewritten, so it is the safe way to undo something that is already shared (pushed to `main`, deployed).

VISUAL

reset (rewrites):

A — B — C

→

A — B

C disappears from the branch

revert (adds):

A — B — C

→

A — B — C — C'

C' = the exact opposite of C
everyone can pull it normally

COMMANDS

Command	What it does
<code>git revert C</code>	new commit undoing C (opens the editor for the message)
<code>git revert --no-edit C</code>	keep the default message <code>Revert "..."</code>
<code>git revert --no-commit C1 C2</code>	undo several, commit once yourself
<code>git revert -m 1 MERGE</code>	undo a merge commit, keeping parent 1 (<code>main</code>)
<code>git revert --abort / --continue</code>	when the revert conflicts

RECAP

- `revert` undoes a commit by adding its opposite; history is kept.
- Use `revert` for anything already pushed; `reset` only for local, unshared commits.
- Reverts can conflict: resolve, `git add`, `git revert --continue`.

Full lesson with every command and its real output: [32-git-revert](#)

Lesson 33 · git reflog

The reflog is your local diary of every position `HEAD` and each branch have had: commits, resets, checkouts, merges, rebases. Almost anything you "lost" with Git is still in it.

VISUAL

```
git log      = the history of commits reachable from the branch    (what the project looks like)
git reflog   = the history of where HEAD has been, in this clone   (what YOU did)

HEAD@{0}  reset: moving to HEAD~3          ← now
HEAD@{1}  commit: Price mocha              ← "before my mistake": still here!
HEAD@{2}  commit: Add mocha
...
entries expire after 90 days (30 for unreachable commits); the reflog is never pushed.
```

COMMANDS

Command	What it shows / does
<code>git reflog</code>	where HEAD has been (<code>git reflog show HEAD</code>)
<code>git reflog show main</code>	where the branch <code>main</code> has been
<code>git log -g --oneline</code>	the reflog as a log
<code>HEAD@{2}</code>	HEAD two moves ago
<code>main@{yesterday}</code> , <code>main@{1.hour.ago}</code>	where main was at a time
<code>git reset --hard HEAD@{N}</code>	move back to that position
<code>git branch NAME HEAD@{N}</code>	rescue that position into a new branch (safer)

RECAP

- The reflog records every move of HEAD and branches in your clone.
- `HEAD@{N}` / `branch@{N}` name those old positions; reset or branch to them.
- It is local and expires (90 days by default): recover soon, and it never replaces pushing or backups.

Full lesson with every command and its real output: [33-git-reflog](#)

Module 08 · Stash

Lesson 34 · What is a stash?

The stash is a shelf for unfinished work. `git stash` puts your uncommitted changes aside and gives you a clean working directory, so you can switch to something urgent and come back later.

VISUAL



COMMANDS

Command	What it does
<code>git stash</code>	shelve tracked changes (staged and unstaged); clean working directory
<code>git stash list</code>	what is on the shelf
<code>git stash pop</code>	re-apply the latest stash and remove it from the shelf
<code>git stash -u</code>	also stash untracked (new) files

RECAP

- `git stash` shelves uncommitted work and cleans the working directory; `git stash pop` brings it back.
- Untracked files need `git stash -u`.
- Stashes are local commits under `refs/stash`; they are never pushed.

Full lesson with every command and its real output: [34-what-is-stash](#)

Lesson 35 · Stash, apply and pop

The difference between `apply` (re-apply, keep the stash) and `pop` (re-apply and delete it), how to give stashes names, and what happens when a stash conflicts with newer commits.

VISUAL

```
stash@{0}  — git stash apply —> changes back in the working directory; stash@{0} STAYS on the shelf
stash@{0}  — git stash pop   —> changes back in the working directory; stash@{0} REMOVED
                                     (if the apply conflicts, pop keeps the stash, just in case)
```

COMMANDS

Command	What it does
<code>git stash push -m "msg"</code>	stash with a description
<code>git stash push FILE</code>	stash only FILE
<code>git stash apply [stash@{N}]</code>	re-apply, keep the stash
<code>git stash pop [stash@{N}]</code>	re-apply and remove (kept if it conflicts)
<code>git stash apply --index</code>	also restore which changes were staged
<code>git stash drop [stash@{N}]</code>	delete one stash

RECAP

- `apply` keeps the stash, `pop` removes it (unless it conflicts).
- Name stashes with `-m`; stash single files with `git stash push FILE`.
- Stash conflicts are resolved like merge conflicts; then `git stash drop`.

Full lesson with every command and its real output: [35-stash-apply-pop](#)

Lesson 36 · Managing stashes

How to keep several stashes under control: list them, see what is inside, apply a specific one, turn one into a branch, clean up, and recover a stash dropped by mistake.

VISUAL

```
stash@{0} On main: menu ideas      ← newest (always @{0})
stash@{1} On main: price draft
stash@{2} WIP on main: 4267004 ... ← oldest

git stash show -p stash@{1}      look inside
git stash apply stash@{1}        use it
git stash branch NAME stash@{1}  new branch from where it was made + apply + drop
git stash drop stash@{1}         delete one      git stash clear: delete ALL
```

COMMANDS

Command	What it does
<code>git stash list</code>	all stashes, newest first
<code>git stash show [-p] stash@{N}</code>	files changed / full diff
<code>git stash apply stash@{N}</code>	apply a specific stash
<code>git stash branch NAME [stash@{N}]</code>	new branch from the stash's base, apply, drop
<code>git stash drop stash@{N}</code>	delete one
<code>git stash clear</code>	delete all (careful)

RECAP

- `git stash list` / `show -p` to know what you have; apply a specific `stash@{N}`.
- `git stash branch` turns a stash into a branch, avoiding conflicts with a moved `main`.
- Dropped stashes are unreachable commits: `git fsck --unreachable` can still find them for a while.

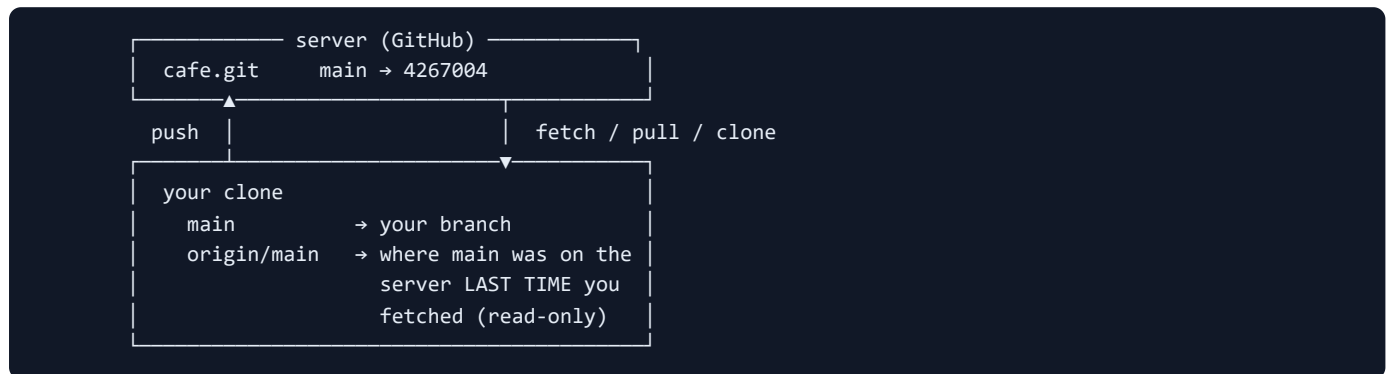
Full lesson with every command and its real output: [36-managing-stashes](#)

Module 09 · Remote repositories

Lesson 37 · What is a remote?

A remote is another copy of the repository that yours knows by a short name, usually `origin`. We look at how your clone remembers the remote's branches (`origin/main`) and why they are only a snapshot of the last time you talked to it.

VISUAL



In this course the "server" is a bare repository on your disk (`server/cafe.git`), which behaves exactly like GitHub for Git's purposes; module 10 uses the real GitHub.

COMMANDS

Command	What it shows
<code>git remote</code>	remote names
<code>git remote -v</code>	names with URLs
<code>git remote show origin</code>	details: branches, tracking, push/pull config (contacts the server)
<code>git branch -r</code>	remote-tracking branches
<code>git branch -a</code>	local and remote-tracking branches

RECAP

- A remote is a named URL of another copy; `origin` is the default name after cloning.
- `origin/main` is a local, read-only snapshot of the server's `main`, updated only by fetch/pull/push.
- `git ls-remote` asks the server live.

Full lesson with every command and its real output: [37-what-is-remote](#)

Lesson 38 · git remote

Managing remotes: add, rename, change the URL, remove. And working with two remotes, the common `origin` + `upstream` setup of a fork.

VISUAL

```
upstream (the original project, read-only for you)
  | fetch
  ▼
your clone —push→ origin (your fork, or the team repository)

git remote add NAME URL      git remote rename OLD NEW
git remote set-url NAME URL   git remote remove NAME
```

COMMANDS

Command	What it does
<code>git remote add NAME URL</code>	add a remote
<code>git remote set-url NAME URL</code>	change its address (e.g. HTTPS → SSH, lesson 49)
<code>git remote rename OLD NEW</code>	rename it (remote-tracking branches follow)
<code>git remote remove NAME</code>	remove it and its remote-tracking branches
<code>git remote get-url NAME</code>	print the URL

RECAP

- `git remote add / set-url / rename / remove` manage the list of remotes.
- One `origin`; change its URL instead of adding it again.
- Forks usually have `origin` (your fork) and `upstream` (the original).

Full lesson with every command and its real output: [38-git-remote](#)

Lesson 39 · git clone

`git clone URL` copies a whole repository (every commit, every branch) and sets up `origin` and the local `main`. We also see shallow clones and cloning a specific branch, both common in CI.

VISUAL

```
git clone URL [FOLDER]
1. create FOLDER, git init
2. git remote add origin URL
3. git fetch origin          ← all commits and branches → origin/*
4. create local main from origin/main and check it out (main tracks origin/main)
```

COMMANDS

Command	What it does
<code>git clone URL</code>	clone into a folder named after the repository
<code>git clone URL DIR</code>	clone into DIR
<code>git clone --branch B URL</code>	check out B instead of the default branch
<code>git clone --depth 1 URL</code>	shallow: only the latest commit (fast, for CI)
<code>git clone --single-branch --branch B URL</code>	only fetch branch B

RECAP

- `git clone` = init + add origin + fetch everything + check out the default branch.
- Remote branches arrive as `origin/*`; `git switch NAME` creates the local tracking branch.
- `--depth 1` for fast CI clones; `--unshallow` to get the history later.

Full lesson with every command and its real output: [39-git-clone](#)

Lesson 40 · git fetch

`git fetch` downloads new commits from the remote and updates `origin/*`, **without touching your branches or files**. It is the safe way to see what changed before you integrate it.

VISUAL

before fetch: main, origin/main → B server: main → D (Grace pushed C and D)

git fetch: downloads C, D; moves origin/main → D
 main stays at B; your files are unchanged

```

A — B ← main
   |
   C — D ← origin/main    "behind 2"
```

COMMANDS

Command	What it does
<code>git fetch</code>	fetch from <code>origin</code> (all branches)
<code>git fetch REMOTE / --all</code>	a specific remote / all remotes
<code>git fetch --prune</code>	also delete <code>origin/*</code> branches removed on the server
<code>git log main..origin/main</code>	incoming commits
<code>git log origin/main..main</code>	outgoing commits (yours, not pushed)
<code>git status</code>	ahead/behind summary (after a fetch)

RECAP

- `git fetch` updates `origin/*` only; your branches and files are untouched.
- `main..origin/main` incoming, `origin/main..main` outgoing.
- `--prune` (or `fetch.prune true`) removes branches deleted on the server.

Full lesson with every command and its real output: [40-git-fetch](#)

Lesson 41 · git pull

`git pull` = `git fetch` + integrate (merge or rebase) `origin/<branch>` into the current branch. We see the easy case, the "divergent branches" error, and how to configure the behaviour you want.

VISUAL

```
git pull = git fetch + git merge origin/main    (pull.rebase false)
              or git rebase origin/main        (pull.rebase true)
              or fast-forward only              (pull.ff only)

diverged:  A — B — X      ← main (your commit X)
           |
           C              ← origin/main (Grace's C)
Git asks you to choose: merge (M joins X and C) or rebase (X' replayed after C)
```

COMMANDS

Command	What it does
<code>git pull</code>	fetch + integrate according to configuration
<code>git pull --ff-only</code>	only if no merge is needed
<code>git pull --no-rebase</code>	fetch + merge
<code>git pull --rebase</code>	fetch + rebase your local commits on top
<code>git config pull.rebase true / pull.ff only</code>	set the default

RECAP

- `git pull` = fetch + merge or rebase.
- Diverged branches need a decision: `--no-rebase` (merge commit) or `--rebase` (linear history).
- Configure `pull.rebase` / `pull.ff` once; use `--ff-only` on shared branches.

Full lesson with every command and its real output: [41-git-pull](#)

Lesson 42 · git push

`git push` uploads your commits and moves the branch on the server. It only works as a fast-forward; when someone pushed before you, it is rejected. We see why, the right fix, and the safe way to force-push your own branch.

VISUAL

```
server main: A — B — C   (Grace's C)
your main:   A — B — X   (your X)

git push → rejected: moving the server's main to X would throw away C
fix:      git pull (merge/rebase) → A — B — C — X' → git push ✓

--force           overwrite whatever is there (C is lost on the server!)
--force-with-lease overwrite only if the server still has what I last fetched
```

COMMANDS

Command	What it does
<code>git push</code>	push the current branch to its upstream (lesson 43)
<code>git push origin BRANCH</code>	push BRANCH to origin
<code>git push -u origin BRANCH</code>	push and set the upstream
<code>git push origin --delete BRANCH</code>	delete a branch on the server
<code>git push --tags / origin TAG</code>	push tags (lesson 69)
<code>git push --force-with-lease</code>	replace the remote branch, only if nobody else changed it

RECAP

- A push must fast-forward the server's branch; otherwise it is rejected.
- Fix a rejection with `git pull` (merge or rebase), then push again.
- Only rewrite pushed history on your own branch, with `--force-with-lease`.

Full lesson with every command and its real output: [42-git-push](#)

Lesson 43 · Upstream branches

The upstream (tracking) branch is the remote branch a local branch is connected to. It is what lets you type plain `git push`, `git pull` and see "ahead 1, behind 2" in `git status`.

VISUAL

```
local branch      upstream
main              →  origin/main          set automatically by clone / git switch NAME
feature-chai     →  origin/feature-chai   set with git push -u origin feature-chai

git branch -vv:
* feature-chai 1a2b3c4 [origin/feature-chai: ahead 1] Add chai
main          4267004 [origin/main] Add prices
```

COMMANDS

Command	What it does
<code>git branch -vv</code>	each branch with its upstream and ahead/behind
<code>git push -u origin BRANCH</code>	push and set the upstream
<code>git branch -u origin/BRANCH</code>	set the upstream of the current branch
<code>git branch --unset-upstream</code>	remove it
<code>@{u} / @{upstream}</code>	"my upstream" in any command: <code>git log @{u}..</code>
<code>git config --global push.autoSetupRemote true</code>	<code>git push</code> sets the upstream by itself the first time

RECAP

- The upstream connects a local branch to a remote branch: plain push/pull and ahead/behind use it.
- `git push -u origin BRANCH` the first time (or `push.autoSetupRemote true`).
- Check with `git branch -vv`; fix with `git branch -u`.

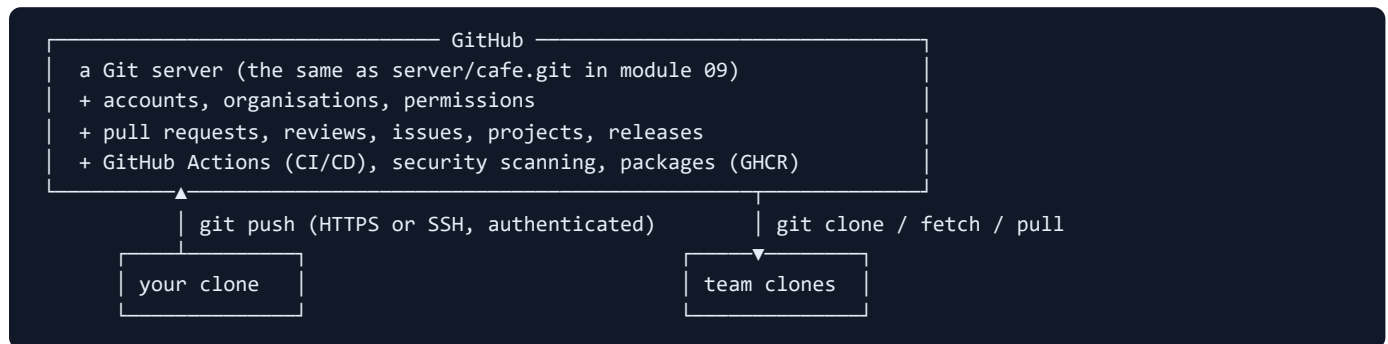
Full lesson with every command and its real output: [43-upstream-branches](#)

Module 10 · GitHub fundamentals

Lesson 44 · What is GitHub?

GitHub is a hosting and collaboration platform built around Git repositories. Git does the version control; GitHub adds a shared server, access control, pull requests, reviews, issues, Actions and releases on top. We talk to a real GitHub repository with plain Git.

VISUAL



GitLab, Bitbucket and Azure DevOps offer the same idea; the Git commands are identical.

COMMANDS

Command	What it does
<code>git ls-remote URL</code>	list a remote repository's branches and tags
<code>git clone https://github.com/OWNER/REPO</code>	clone over HTTPS
<code>git clone git@github.com:OWNER/REPO.git</code>	clone over SSH (lesson 49)
<code>gh repo view OWNER/REPO</code>	the GitHub CLI: description, README, metadata

RECAP

- GitHub = a Git server + collaboration features (PRs, issues, Actions, releases).
- All Git commands work with GitHub URLs exactly as with any remote.
- A credential prompt for a "public" repository usually means a wrong name or no access.

Full lesson with every command and its real output: [44-what-is-github](#)

Lesson 45 · Creating a GitHub repository

How to create a repository on GitHub, in the web interface and with the GitHub CLI, and which options matter when you are going to push an existing project into it.

This lesson and the following GitHub lessons need a free GitHub account. The practice repository is called `git-practice-cafe`; you can delete it at the end of the course.

VISUAL

```
github.com → "+" (top right) → New repository
Owner / Repository name      your-account / git-practice-cafe
Description                  Practice repository for the Git course
Public | Private
Initialize with: README / .gitignore / license
└─ leave all EMPTY when you will push an existing local repository
   (otherwise GitHub creates a first commit, and your push meets "unrelated histories", lesson 25)
[Create repository] → a page with the URL and "...or push an existing repository from the command line"
```

COMMANDS

Command / option	Meaning
<code>gh repo create NAME --public / --private</code>	create under your account
<code>gh repo create ORG/NAME</code>	create in an organisation
<code>--add-readme</code> , <code>--gitignore Go</code> , <code>--license mit</code>	initialise with files (a first commit)
<code>--source .</code> <code>--push</code>	create from the current local repository and push it
<code>gh repo view NAME</code>	show it; <code>--web</code> opens the browser
<code>gh repo delete NAME</code>	delete (needs the <code>delete_repo</code> permission)

RECAP

- Create empty repositories when you will push an existing project.
- `gh repo create` does the same as the web form; names are unique per owner.
- Organisations and templates are the usual way in teams.

Full lesson with every command and its real output: [45-create-github-repository](#)

Lesson 46 · Connecting local Git to GitHub

How to push an existing local repository into the empty GitHub repository from lesson 45: add the remote, push with upstream. Exactly what GitHub's "...or push an existing repository from the command line" box tells you.

VISUAL

```
local repository (3 commits)      github.com/YOU/git-practice-cafe (empty)
main ───────────────────────────> main
    git remote add origin URL
    git push -u origin main
```

COMMANDS

Command	What it does
<code>git remote add origin URL</code>	connect the local repository to GitHub
<code>git push -u origin main</code>	upload <code>main</code> , set its upstream
<code>git branch -M main</code>	rename the current branch to <code>main</code> (GitHub's snippet includes it for old <code>master</code> setups)
<code>gh repo create NAME --source . --push</code>	create the GitHub repository and push in one step

RECAP

- `git remote add origin URL` + `git push -u origin main` publishes a local repository.
- An authenticated `Repository not found` means a wrong owner/name or no access.
- The commit IDs are identical locally and on GitHub.

Full lesson with every command and its real output: [46-connect-local-to-github](#)

Lesson 47 · GitHub repository structure

What each tab of a GitHub repository page is for, which of them are plain Git (Code, Branches, Tags) and which are GitHub features stored outside Git (Issues, Pull requests, Actions, Releases, Settings). For each tab we use the command-line equivalent.

VISUAL

github.com/YOU/git-practice-cafe

Code	Issues	Pull requests	Actions	Projects	Settings
------	--------	---------------	---------	----------	----------

Code: files of the default branch, README, branch selector, "N branches", "N tags", Releases (right side)

stored IN Git (cloned with the repository)
files, commits, branches, tags

stored by GitHub (NOT in a clone)
issues, PRs + reviews, Actions runs,
releases' notes and assets, settings

COMMANDS

Tab	Web	CLI
Code	file browser, README	git clone , gh repo view
Branches / Tags	/branches , /tags	git ls-remote , git branch -r , git tag
Issues	/issues	gh issue list / create / view (lesson 93)
Pull requests	/pulls	gh pr list / create / review / merge (lesson 52)
Actions	/actions	gh run list / view / watch (lesson 98)
Releases	/releases	gh release list / create (lesson 97)
Settings	/settings	gh repo edit , gh api repos/OWNER/REPO

RECAP

- Code, branches and tags are Git; issues, PRs, Actions, releases and settings are GitHub.
- A clone contains only the Git part.
- `gh` is the command-line window to the GitHub part (`-R OWNER/REPO` outside a clone).

Full lesson with every command and its real output: [47-github-repository-structure](#)

Lesson 48 · HTTPS authentication

How Git proves who you are to GitHub over HTTPS: not with your account password (GitHub stopped accepting it for Git in 2021), but with a token, handed to Git by a **credential helper** so you never type it.

VISUAL

```
git push https://github.com/YOU/repo.git
|
| 401: who are you?
|
└─ credential helper → returns username + TOKEN (stored in the OS keychain / gh's login)
                        Git Credential Manager (Windows, macOS, Linux), osxkeychain, gh auth git-credential
                        |
                        └─ GitHub checks the token: valid? not expired? allowed for this repository (scopes / fine-grained permissions)?
```

Credential	Use
Account password	the website only (with 2FA); never accepted by Git
Personal access token (fine-grained)	Git and API, limited to chosen repositories and permissions, with an expiry
gh auth login / Git Credential Manager	log in once in the browser; the token is stored and handed to Git
GITHUB_TOKEN	the automatic, short-lived token inside GitHub Actions

COMMANDS

Command	What it does
gh auth login	log in to GitHub in the browser, store a token
gh auth setup-git	make Git use gh's token for github.com
git config --global credential.helper manager	Git Credential Manager (installed with Git for Windows)
git config --show-origin --get-regexp '^credential'	which helper is configured, and where
gh auth status	which account, which token scopes

RECAP

- GitHub never accepts your account password for Git; HTTPS needs a token.
- A credential helper (Git Credential Manager, `gh auth setup-git`) stores and supplies it.
- Prefer fine-grained, expiring, least-privilege tokens; in Actions use `GITHUB_TOKEN`.

Full lesson with every command and its real output: [48-https-authentication](#)

Lesson 49 · SSH authentication

How to authenticate to GitHub with an SSH key pair: create the key with `ssh-keygen`, trust GitHub's host key, add the public key to your account, and verify with `ssh -T git@github.com`.

VISUAL

```
your computer                                GitHub
~/.ssh/id_ed25519      (PRIVATE: never leaves)
~/.ssh/id_ed25519.pub  (public) — added once to Settings → SSH and GPG keys
~/.ssh/known_hosts     GitHub's host key ← checked on every connection: "am I talking to the real GitHub?"

ssh -T git@github.com
  1. GitHub proves its identity (host key, compared with known_hosts)
  2. you prove yours: a signature made with the private key, checked against your registered public key
  3. "Hi YOU! You've successfully authenticated, but GitHub does not provide shell access."
```

COMMANDS

Command	What it does
<code>ssh-keygen -t ed25519 -C "email"</code>	create a key pair (asks for file and passphrase)
<code>ssh-keygen -l -f FILE</code>	show a key's fingerprint
<code>ssh-add ~/.ssh/id_ed25519</code>	load the key into the agent (passphrase typed once per session)
<code>gh ssh-key add FILE.pub --title NAME</code>	register the public key on GitHub (or: Settings → SSH and GPG keys)
<code>ssh -T git@github.com</code>	test: who does GitHub think I am?
<code>ssh -vT git@github.com</code>	the same, with a debug log of every step

RECAP

- `ssh-keygen -t ed25519` creates the pair; only the `.pub` part goes to GitHub.
- Verify GitHub's host key fingerprint once; `known_hosts` remembers it.
- `ssh -T git@github.com` tests; `Permission denied (publickey)` = no registered key was offered.

Full lesson with every command and its real output: [49-ssh-authentication](#)

Lesson 50 · SSH vs HTTPS

Both protocols reach the same repository. We compare them in practice, switch a clone from one to the other, and fix the most common network problem: port 22 blocked by a firewall.

VISUAL

	HTTPS	SSH
URL	https://github.com/YOU/repo.git	git@github.com:YOU/repo.git
port	443 (open almost everywhere)	22 (sometimes blocked) · or 443 via
ssh.github.com		
credential	token via a credential helper	key pair (+ passphrase, ssh-agent)
set up once	gh auth login / Git Credential Manager	ssh-keygen + add the public key
expires	tokens: yes (by design)	keys: no (unless you rotate them)
scope	token permissions (fine-grained)	full access of the account (or one repo:
deploy key)		
also used for	the GitHub API (gh, curl)	commit signing (lesson 91)

Neither is "more secure" by itself: a fine-grained expiring token and a passphrase-protected key are both good choices. Pick one per computer and be consistent.

COMMANDS

Command	What it does
<code>git remote set-url origin git@github.com:O/R.git</code>	HTTPS → SSH for one clone
<code>git config --global url."git@github.com:".insteadOf https://github.com/</code>	rewrite for every clone
<code>ssh -T -p 443 git@ssh.github.com</code>	test SSH over port 443
<code>Host github.com / Hostname ssh.github.com / Port 443</code>	make that permanent in <code>~/.ssh/config</code>

RECAP

- Same repository, two transports: HTTPS (port 443, tokens) and SSH (port 22 or 443, keys).
- `git remote set-url` or `url.<base>.insteadOf` switch between them.
- Timeouts = network; `Permission denied (publickey)` = authentication.

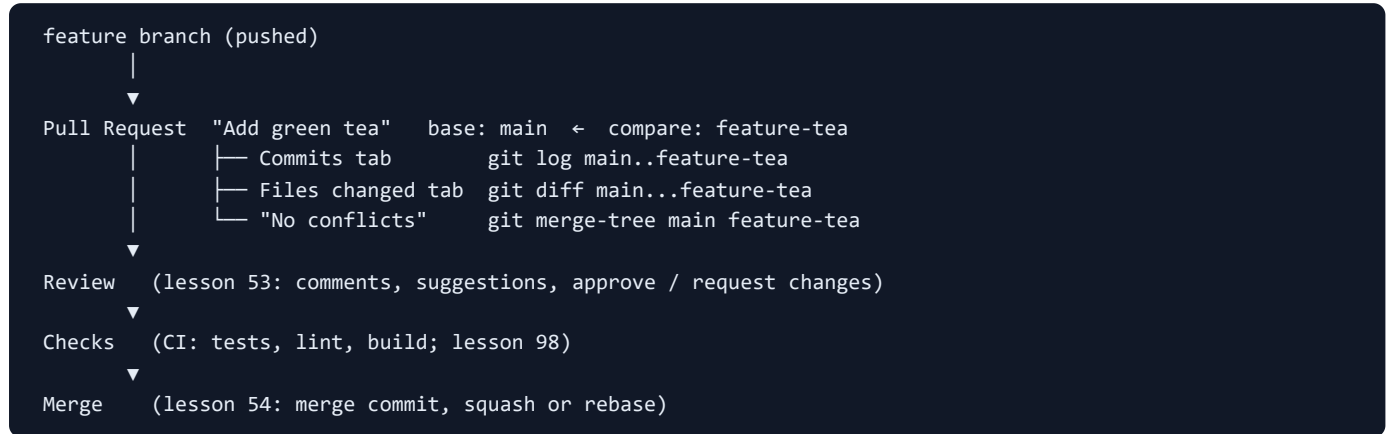
Full lesson with every command and its real output: [50-ssh-vs-https](#)

Module 11 · Pull requests

Lesson 51 · What is a pull request?

A pull request (PR) is a request to merge one branch into another, wrapped in a page for discussion: the commits, the diff, a merge check, reviews and automated checks. Underneath it is plain Git, which we reproduce locally first.

VISUAL



COMMANDS

PR page	Git equivalent
Commits	<code>git log BASE..HEAD-BRANCH</code>
Files changed	<code>git diff BASE...HEAD-BRANCH</code> (three dots)
"No conflicts" / "conflicts must be resolved"	<code>git merge-tree --write-tree BASE BRANCH</code> (exit status 0 / 1)
Merge button	<code>git merge</code> (or squash / rebase, lesson 54) executed by GitHub

RECAP

- A PR = a branch comparison + discussion + checks + a merge button.
- Commits tab = `base..branch` ; Files changed = `base...branch` ; conflicts = `git merge-tree` .
- Conflicts are fixed on the branch by its author; the PR updates automatically when the branch is pushed.

Full lesson with every command and its real output: [51-what-is-pull-request](#)

Lesson 52 · Creating a pull request

The full loop on the real GitHub: branch, commit, push, open a PR with a useful title and description, update it with more commits. We use the GitHub CLI; the web flow is shown alongside.

VISUAL

```
git switch -c add-green-tea → commit → git push -u origin add-green-tea
|
▼ GitHub shows a yellow banner: "add-green-tea had recent pushes [Compare & pull request]"
gh pr create --base main --title "..." --body "..."
|
▼
PR #N open — new commits pushed to add-green-tea appear in the PR automatically
```

COMMANDS

Command	What it does
<code>gh pr create --base B --title T --body X</code>	open a PR from the current branch into B
<code>gh pr create --fill</code>	title/body from the commit messages
<code>gh pr create --draft</code>	a draft: visible, not ready for review or merging
<code>gh pr view [N BRANCH] [--web]</code>	details (or open the page)
<code>gh pr diff / gh pr checks</code>	the diff / the CI status
<code>gh pr edit --add-reviewer X --add-label Y</code>	change reviewers, labels, title, body
<code>gh pr close N --delete-branch</code>	close without merging

RECAP

- Push the branch, then `gh pr create` (or the "Compare & pull request" button).
- New commits on the branch update the PR; one open PR per branch and base.
- Write the why, what, how-tested in the description; drafts for early feedback.

Full lesson with every command and its real output: [52-create-pull-request](#)

Lesson 53 · Pull request review

How a review works on GitHub: general comments, comments on a line, **suggestions** the author can apply with one click, and the three review verdicts (comment, approve, request changes). And why you cannot approve your own PR.

VISUAL

```
PR #N — Files changed — line 4 of prices.txt
├── comment      "Is 2.80 the price with tax?"
├── suggestion   ```suggestion
│               green tea 2.90
│               ```
│               → author clicks "Commit suggestion"
└── conversation replies ... → "Resolve conversation"

Submit review:  o Comment    o Approve  o Request changes
                  (no verdict) (ready)   (blocks merging while protection requires reviews)
```

COMMANDS

Command	What it does
<code>gh pr review N --comment -b TEXT</code>	review without a verdict
<code>gh pr review N --approve [-b TEXT]</code>	approve
<code>gh pr review N --request-changes -b TEXT</code>	request changes
<code>gh pr comment N -b TEXT</code>	a conversation comment (not a review)
<code>gh api repos/O/R/pulls/N/comments -f path=... -F line=...</code>	a line comment (with an optional ```suggestion block)
<code>gh pr checkout N</code>	check the PR's branch out locally to test it

RECAP

- Review verdicts: comment, approve, request changes; line comments can carry one-click suggestions.
- The author answers by pushing commits; the PR updates.
- Nobody can approve their own PR: required reviews need a second person.

Full lesson with every command and its real output: [53-pull-request-review](#)

Lesson 54 · Merge strategies

GitHub's merge button has three modes: **Create a merge commit**, **Squash and merge**, **Rebase and merge**. Each leaves a different history on `main`. We reproduce all three locally to see the graphs, then merge a real PR.

VISUAL

PR branch: main: A — B feature: B — f1 — f2

merge commit A — B ——— M
 \ /
 f1 — f2

all commits kept + a merge commit (the PR is visible as a bubble)

squash and merge A — B — S
rebase and merge A — B — f1' — f2'

ONE new commit S = f1 + f2 (branch commits not on main)
commits replayed one by one, new IDs, no merge commit

COMMANDS

GitHub button	Local equivalent	History on main	Use when
Create a merge commit	<code>git merge --no-ff BRANCH</code>	all commits + merge commit	commits are meaningful; you want the PR boundary
Squash and merge	<code>git merge --squash BRANCH</code> && <code>git commit</code>	one commit per PR	messy branch history; one PR = one change
Rebase and merge	<code>git rebase main + fast-forward</code>	all commits, linear	clean, meaningful commits; linear history
<code>gh pr merge N --merge / --squash / --rebase</code>			CLI

RECAP

- Merge commit: keeps everything + the PR boundary; squash: one commit per PR; rebase: linear, all commits.
- After squash/rebase merges, `git branch -d` says "not fully merged": compare content, then `-d`.
- Configure the allowed strategies per repository.

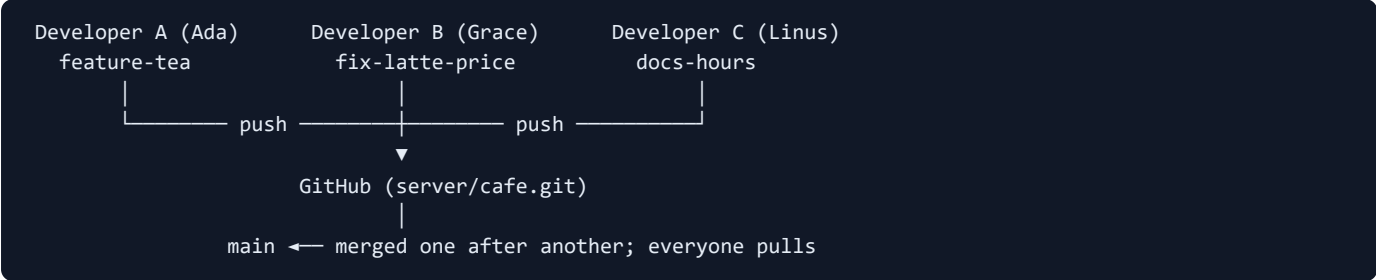
Full lesson with every command and its real output: [54-merge-strategies](#)

Module 12 · Collaboration

Lesson 55 · Team Git workflow

What happens when three people work on the same repository at the same time, and the small set of habits that keeps it calm: short branches, pull before you start, push often, integrate through the shared server.

VISUAL



COMMANDS

Habit	Commands
start from the latest main	<code>git switch main && git pull</code>
one branch per task	<code>git switch -c TASK</code>
push early (backup, visibility)	<code>git push -u origin TASK</code>
integrate through PRs	<code>gh pr create</code> , review, merge
clean up	<code>git branch -d TASK</code> , <code>git fetch --prune</code>

RECAP

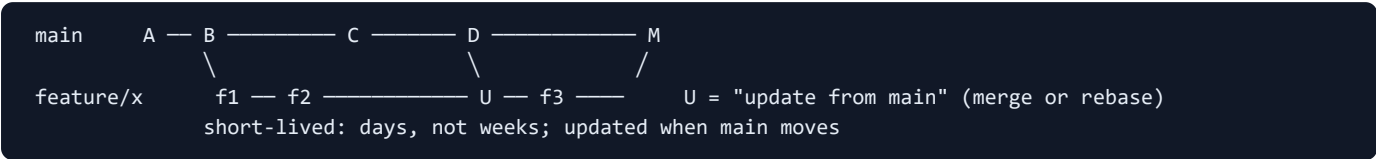
- Many developers, one server: integrate through it, in small pieces.
- Pull before starting, branch per task, push early, merge via PRs.
- Direct commits to a shared `main` cause rejected pushes and conflicts.

Full lesson with every command and its real output: [55-team-workflow](#)

Lesson 56 · Feature branch workflow

The feature branch workflow in detail: naming, keeping the branch up to date with `main` while it lives, and why long-lived branches hurt.

VISUAL



Naming conventions (pick one per team): `feature/menu-tea`, `fix/latte-price`, `docs/opening-hours`, `chore/update-chart`, optionally with an issue number: `feature/42-menu-tea`.

COMMANDS

Step	Commands
start	<code>git switch main && git pull && git switch -c feature/NAME</code>
save + share	<code>git commit</code> , <code>git push -u origin feature/NAME</code>
update from main	<code>git fetch && git merge origin/main</code> (or <code>git rebase origin/main</code> , lesson 61)
finish	PR → review → merge → <code>git switch main && git pull && git branch -d feature/NAME</code>

RECAP

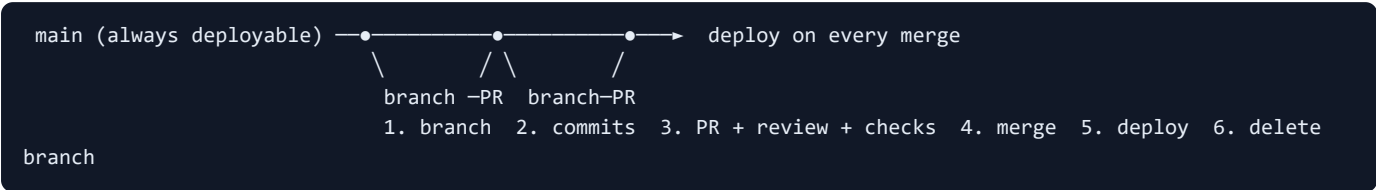
- One short-lived branch per task, named by convention.
- Update the branch from `main` regularly; conflicts stay small.
- Long-lived branches = big conflicts; prefer small PRs and feature flags.

Full lesson with every command and its real output: [56-feature-branch-workflow](#)

Lesson 57 · GitHub Flow

GitHub Flow is the simplest common team workflow: `main` is always deployable; every change goes through a short branch and a pull request; what is merged is deployed. We run it end to end with a tiny "deployment".

VISUAL



COMMANDS

Rule of GitHub Flow	In practice
<code>main</code> is always deployable	protect it (lesson 59); CI on every PR
branch from <code>main</code> , descriptive names	<code>git switch -c add-chai</code>
open a PR early	draft PRs for discussion (lesson 52)
merge only after review and green checks	branch protection rules
deploy right after merging	CD triggered by pushes to <code>main</code> (lesson 98)

RECAP

- GitHub Flow: `main` always deployable; branch → PR → review/checks → merge → deploy.
- Bypassing review and checks breaks production for everyone.
- Fix forward through the same flow: a revert PR.

Full lesson with every command and its real output: [57-github-flow](#)

Lesson 58 · Git Flow

Git Flow is a branching model with two long-lived branches (`main` for releases, `develop` for integration) and short-lived `feature/*`, `release/*` and `hotfix/*` branches. We run a full cycle with plain Git commands and discuss where it helps and where it is unnecessary complexity.

VISUAL



COMMANDS

Branch	From	Into	Lifetime
<code>main</code>	—	—	permanent; every commit is a release
<code>develop</code>	<code>main</code>	—	permanent; next release in progress
<code>feature/*</code>	<code>develop</code>	<code>develop</code>	days
<code>release/*</code>	<code>develop</code>	<code>main</code> + <code>develop</code>	until the release ships
<code>hotfix/*</code>	<code>main</code>	<code>main</code> + <code>develop</code>	hours

RECAP

- Git Flow: `main` (releases) + `develop` (integration) + feature, release, hotfix branches.
- Hotfixes go into `main` **and** `develop`; check with `git log develop..main`.
- Good for versioned products; often unnecessary for continuously deployed services.

Full lesson with every command and its real output: [58-git-flow](#)

Lesson 59 · Branch protection

How a server enforces the team rules: no direct pushes to `main`, changes only through pull requests, required reviews, required status checks, no force pushes. First we build the mechanism ourselves with a server-side hook, then we use GitHub's branch protection on the practice repository.

VISUAL

```
git push origin main → server → protection rule for main:  
    X direct push           → rejected  
    X force push / deletion → rejected  
    ✓ merge of a PR with ≥1 approval and green checks
```

On GitHub: repository → Settings → Branches → "Add branch protection rule" (or Settings → Rules → Rulesets).

COMMANDS

GitHub protection setting	Effect
Require a pull request before merging	no direct pushes to the branch
Required approvals: 1	someone other than the author must approve (lesson 53)
Require status checks to pass	CI must be green (lesson 98)
Require branches to be up to date	the PR must contain the latest <code>main</code>
Do not allow bypassing (enforce for admins)	the rules apply to administrators too
Allow force pushes / deletions: off	history of <code>main</code> cannot be rewritten or deleted

RECAP

- Protection is enforced by the server at push time (GitHub rules ≈ a managed `pre-receive` hook).
- Require PRs, approvals and checks; disallow force pushes; enforce for admins.
- A rejected push changes nothing on the server: move the commit to a branch and open a PR.

Full lesson with every command and its real output: [59-branch-protection](#)

Module 13 · Rebase

Lesson 60 · What is rebase?

Rebase **replays** your branch's commits on top of another commit, as if you had started your work later. The result is a straight history, but the replayed commits are **new commits** with new IDs. Understanding that one fact explains every rule about rebase.

VISUAL

```
before:                                git rebase main (on feature-tea)

A — B — D                            A — B — D
 |                                     |
C                                     C'
← main                               ← main
← feature-tea                       ← feature-tea

C' has the same change and message as C, but a different parent (D instead of B),
so a different ID. C still exists (reflog), but nothing points to it any more.
```

How Git does it, step by step:

```
1. find the commits on feature-tea that are not on main      → C
2. move feature-tea to main's tip                          → D
3. re-apply each of those commits, in order, as new commits  → C'
```

COMMANDS

Command	What it does
<code>git rebase BASE</code>	replay the current branch's own commits on top of BASE
<code>git rebase BASE BRANCH</code>	switch to BRANCH first, then rebase it
<code>git rebase -i BASE</code>	interactive: choose what happens to each commit (lesson 63)
<code>git pull --rebase</code>	fetch + rebase your local commits on the remote branch

RECAP

- Rebase replays commits on a new base: same changes, new commits, new IDs.
- Linear history, but rewritten history: never rebase what others have built on.
- After rebasing your own pushed branch: `git push --force-with-lease`.

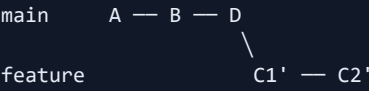
Full lesson with every command and its real output: [60-what-is-rebase](#)

Lesson 61 · Basic rebase

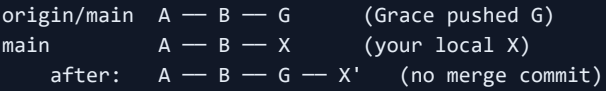
The everyday rebase: bring your feature branch up to date with `main` before opening or updating a PR, and `git pull --rebase` to keep your local commits on top of your teammates' work.

VISUAL

```
git switch feature && git rebase main
```



```
git pull --rebase (on main)
```



COMMANDS

Command	When
<code>git rebase main</code>	update your feature branch with main (local main must be current)
<code>git fetch && git rebase origin/main</code>	the same against the server's main, without updating local main
<code>git pull --rebase</code>	update a branch from its upstream, local commits on top
<code>git rebase --autostash main</code>	stash uncommitted changes first, re-apply after
<code>git config --global pull.rebase true</code>	make <code>git pull</code> rebase by default

RECAP

- `git rebase main` puts your branch's commits on top of main: linear, fast-forwardable.
- `git pull --rebase` does the same against the upstream.
- Commit or `--autostash` uncommitted work before rebasing.

Full lesson with every command and its real output: [61-basic-rebase](#)

Lesson 62 · Rebase vs merge

Merge and rebase both integrate work; they produce different histories. We build both from the same starting point, compare them, and see the one situation where rebase really hurts: rebasing a branch someone else has built on.

VISUAL

same start: main A — B — D

merge: A — B — D ————— M

 \ /

 C1 — C2 —

feature B — C1 — C2

+ true record of what happened, nothing rewritten

- extra merge commits, a busier graph

rebase: A — B — D — C1' — C2'

+ linear, easy to read and bisect

- rewrites commits: never on shared branches

COMMANDS

Situation	Prefer
update your own unpushed / solo branch with main	rebase
integrate a finished feature into main	merge (or squash) via PR
a branch others have pulled or built on	merge
clean up your own commits before review	interactive rebase (lesson 63)
local commits vs teammates' on the same branch	<code>git pull --rebase</code>

RECAP

- Merge records history as it happened; rebase rewrites it into a straight line.
- Same final files, different graphs.
- Rebasing a shared branch duplicates commits for everyone who built on it.

Full lesson with every command and its real output: [62-rebase-vs-merge](#)

Lesson 63 · Interactive rebase

`git rebase -i` opens a to-do list of your commits, one per line, and lets you decide what happens to each: keep, edit the message, stop to change it, combine, or delete. It is how you turn a messy work-in-progress branch into a clean series of commits before review.

VISUAL

```
git rebase -i HEAD~4 opens your editor with:

pick 6833580 Add green tea      pick keep as is
pick 269869e Price green tea   reword keep, but edit the message
pick 2c389c0 Add mocha        edit stop after applying it (amend, split ...)
pick ecff18a Price mocha      squash meld into the previous commit, combine messages
                             fixup meld into the previous commit, discard this message
                             drop remove the commit

save and close → Git replays the list from top (oldest) to bottom (newest)
```

In this course the editor is replaced by a command (`GIT_SEQUENCE_EDITOR`), so every step can be run and checked; in your terminal you edit the words in the file and save.

COMMANDS

To-do command	Effect
<code>pick (p)</code>	use the commit
<code>reword (r)</code>	use it, edit the message
<code>edit (e)</code>	use it, stop for amending (lesson 66)
<code>squash (s)</code>	meld into the previous commit, edit the combined message
<code>fixup (f)</code>	meld into the previous commit, keep the previous message
<code>drop (d)</code> or delete the line	remove the commit
reorder lines	reorder commits
<code>git commit --fixup=SHA</code> + <code>git rebase -i --autosquash</code>	mark a fix for an older commit, squash it automatically

RECAP

- `git rebase -i BASE` lists the commits after BASE; edit the verbs, save, Git replays.
- pick, reword, edit, squash, fixup, drop; reorder by moving lines.
- Only on commits you have not shared (or your own branch, then `--force-with-lease`).

Full lesson with every command and its real output: [63-interactive-rebase](#)

Lesson 64 · Rebase conflicts

A rebase replays commits one at a time, so a conflict can happen at each step. We create one on purpose and resolve it step by step: read the status, fix the file, `git add`, `git rebase --continue`.

VISUAL

```
git rebase main
  replay C1 ✓
  replay C2 ✗ CONFLICT — rebase pauses → fix the file → git add FILE → git rebase --continue
  replay C3 ✓                                     (or git rebase --skip: drop C2 · git rebase --abort: undo
all)
```

note: during a rebase, "ours" = the branch you are rebasing ONTO (main), "theirs" = your commit being replayed.

(the opposite of a merge on your branch)

COMMANDS

Command	Use during a rebase
<code>git status</code>	which commit is being replayed, which files conflict
<code>git diff</code>	the unresolved conflicts
<code>git add FILE</code>	mark FILE resolved
<code>git rebase --continue</code>	commit the resolution, replay the next commit
<code>git rebase --skip</code>	drop the current commit entirely
<code>git rebase --abort</code>	return to the state before the rebase (lesson 65)
<code>git checkout --ours/--theirs FILE</code>	take main's / your commit's version (note the reversed meaning)

RECAP

- A rebase stops at each conflicting commit; `git status` says which one.
- Fix → `git add` → `git rebase --continue`; `--skip` drops the commit, `--abort` cancels.
- During a rebase, "ours" is the new base and "theirs" is your commit.

Full lesson with every command and its real output: [64-rebase-conflicts](#)

Lesson 65 · Aborting a rebase

`git rebase --abort` cancels a rebase in progress and puts the branch back exactly where it was before, however many commits were already replayed. It is the emergency exit that makes rebasing safe to try.

VISUAL

```
git rebase main
  replay C1 ✓   replay C2 ✓   replay C3 X CONFLICT ... "this is getting complicated"
                                     |
                                     └─ git rebase --abort → branch back at C3 (old IDs), as if nothing
happened
```

COMMANDS

Command	What it does
<code>git rebase --abort</code>	stop and restore the branch and working directory as before the rebase
<code>git rebase --quit</code>	stop but leave HEAD where it is now (rarely what you want)
<code>ORIG_HEAD</code>	the branch tip before the rebase started
<code>git reflog</code>	if you already finished a bad rebase: find the pre-rebase position (lesson 33)

RECAP

- `git rebase --abort` restores everything as before the rebase.
- Only one rebase at a time; `git status` tells you if one is in progress.
- After a completed rebase, `ORIG_HEAD` / `reflog` bring the old branch back.

Full lesson with every command and its real output: [65-abort-rebase](#)

Lesson 66 · Continuing a rebase

`git rebase --continue` resumes a paused rebase. A rebase pauses for two reasons: a conflict (lesson 64) or an `edit` stop you asked for. We use `edit` to change an old commit in the middle of the history, and the next commits follow automatically.

VISUAL

```
git rebase -i HEAD~4 with edit 269869e Price green tea
replay "Add green tea"    ✓
replay "Price green tea" ✓ → STOP (edit): change files, git commit --amend
git rebase --continue
replay "Add mocha"       ✓
replay "Price mocha"     ✓ → done
```

COMMANDS

Command	When
<code>git rebase --continue</code>	after resolving a conflict (<code>git add</code>) or after amending at an <code>edit</code> stop
<code>git commit --amend</code>	at an <code>edit</code> stop: change the stopped commit
<code>git rebase --skip</code>	drop the current commit
<code>git rebase --edit-todo</code>	change the remaining to-do list mid-rebase
<code>git reset HEAD~1</code> at an <code>edit</code> stop	split the commit: then several <code>git add</code> / <code>git commit</code> , then continue

RECAP

- A rebase pauses on conflicts and `edit` stops; `git rebase --continue` resumes.
- At an `edit` stop: `--amend` changes the commit, a plain commit adds one.
- Fix a slip with another interactive rebase (`fixup`).

Full lesson with every command and its real output: [66-continue-rebase](#)

Module 14 · Advanced Git

Lesson 67 · Cherry-pick

`git cherry-pick COMMIT` copies the change of one commit onto the current branch, as a new commit. The classic use: an important bug fix was made on a feature branch, and production needs that fix **now**, without the rest of the unfinished feature.

VISUAL

feature

A — B — F1 — FIX — F2

production

B

B — FIX'

F1, F2: unfinished feature work · FIX: an urgent bug fix

`git switch production`

`git cherry-pick FIX`

FIX' = same change, new commit on production; F1, F2 stay behind

COMMANDS

Command	What it does
<code>git cherry-pick C</code>	apply C's change as a new commit on the current branch
<code>git cherry-pick -x C</code>	add "(cherry picked from commit ...)" to the message
<code>git cherry-pick A^..B</code>	a range, oldest first (A included)
<code>git cherry-pick --no-commit C</code>	apply without committing
<code>git cherry-pick --continue / --abort / --skip</code>	after a conflict

RECAP

- `git cherry-pick C` copies one commit's change onto the current branch as a new commit.
- It does not bring the commits C depends on: pick those first, in order.
- Use `-x` for backports; prefer merging when you want everything.

Full lesson with every command and its real output: [67-cherry-pick](#)

Lesson 68 · Git tags

A tag is a name fixed to one commit, usually a version: `v1.0.0`. Unlike a branch, it does not move when you commit. We create tags, list them, check one out, and push them (tags are **not** pushed with branches).

VISUAL

```
A — B — C — D — E ← main (moves with every commit)
      ▲           ▲
    v1.0.0       v1.1.0      (tags stay where they were created)
```

COMMANDS

Command	What it does
<code>git tag NAME [COMMIT]</code>	lightweight tag on HEAD (or COMMIT)
<code>git tag / git tag -l "v1.*"</code>	list (with a pattern)
<code>git push origin NAME</code>	push one tag
<code>git push --tags / git push --follow-tags</code>	push all tags / annotated tags reachable from pushed commits
<code>git tag -d NAME / git push origin --delete NAME</code>	delete locally / on the server
<code>git ls-remote --tags origin</code>	tags on the server

RECAP

- A tag names a commit permanently; branches move, tags do not.
- Tags must be pushed explicitly: `git push origin TAG`.
- Check out a tag to inspect a release (detached HEAD); never move published tags.

Full lesson with every command and its real output: [68-git-tags](#)

Lesson 69 · Annotated tags

Git has two kinds of tags. A **lightweight** tag is just a name for a commit. An **annotated** tag is an object of its own, with a tagger, a date and a message (and optionally a signature, lesson 91). Releases should use annotated tags.

VISUAL

```
lightweight: refs/tags/v0.9.0 → commit fc345e6
annotated:   refs/tags/v1.0.0 → tag object
                                tagger: Ada <ada@...>, date
                                message: "Release 1.0.0 ..."
```

COMMANDS

Command	What it does
<code>git tag -a NAME -m "msg" [COMMIT]</code>	annotated tag
<code>git tag -s NAME -m "msg"</code>	annotated and signed (lesson 91)
<code>git show NAME</code>	tag details + the commit
<code>git cat-file -t NAME</code>	<code>tag</code> (annotated) or <code>commit</code> (lightweight)
<code>git describe</code>	name HEAD relative to the nearest annotated tag: <code>v1.0.0-4-gecfff18a</code>
<code>git push --follow-tags</code>	push commits and the annotated tags pointing at them

RECAP

- Lightweight tag = a name; annotated tag = an object with tagger, date, message.
- Use annotated (or signed) tags for releases; `git describe` relies on them.
- `git cat-file -t TAG` tells the kinds apart.

Full lesson with every command and its real output: [69-annotated-tags](#)

Lesson 71 · Git blame

`git blame FILE` shows, for every line, the commit that last changed it, who made it and when. It answers "why is this line like this?", by leading you to the commit and its message. It is a research tool, not a tool to blame people.

VISUAL

```
git blame prices.txt
4267004 (Ada Lovelace 2026-01-05 09:03) espresso 2.50      ← last changed in "Add prices"
7f3a2c1 (Grace Hopper 2026-01-05 09:05) latte 3.30       ← last changed in "Raise the latte price"
4267004 (Ada Lovelace 2026-01-05 09:03) cappuccino 3.40
    |
    └─ git show 7f3a2c1 → the commit message explains WHY
```

COMMANDS

Command	What it does
<code>git blame FILE</code>	last commit per line
<code>git blame -L 10,20 FILE</code>	only lines 10–20
<code>git blame -w FILE</code>	ignore whitespace-only changes
<code>git blame --ignore-rev C FILE</code>	skip commit C (e.g. a reformatting)
<code>git config blame.ignoreRevsFile .git-blame-ignore-revs</code>	skip the commits listed in a file, always
<code>git log -L 2,2:prices.txt</code>	the full history of a line range

RECAP

- `git blame` = last commit per line; follow it to the commit message and PR for the "why".
- `-L` for a range, `git log -L` for a line's whole history.
- Ignore formatting commits with `--ignore-rev` / `.git-blame-ignore-revs`.

Full lesson with every command and its real output: [71-git-blame](#)

Lesson 72 · Git clean

`git clean` deletes **untracked** files: build output, temporary files, experiments. They are not in Git, so nothing can bring them back. That is why we always preview with `-n` first.

VISUAL

working directory

├ menu.txt

├ notes-draft.txt

├ build/

└ .env

tracked

untracked

untracked dir

ignored

git clean never touches it

git clean -f deletes it

git clean -fd deletes it

only git clean -x deletes it (careful: local secrets, config)

ALWAYS: git clean -n ... (dry run: "Would remove ...") then the same with -f

COMMANDS

Command	Deletes
<code>git clean -n</code>	nothing: lists what <code>-f</code> would delete
<code>git clean -f</code>	untracked files
<code>git clean -fd</code>	untracked files and directories
<code>git clean -fX</code>	only ignored files (rebuild from scratch, keep your new files)
<code>git clean -fdx</code>	everything untracked, ignored included
<code>git clean -fd -e PATTERN</code>	... except PATTERN
<code>git clean -i</code>	interactive menu

RECAP

- `git clean` permanently deletes untracked files; it requires `-f`.
- Always run `-n` first; `-d` for directories; `-x` / `-X` for ignored files.
- Keep templates (`.env.example`) for anything local you might wipe.

Full lesson with every command and its real output: [72-git-clean](#)

Lesson 73 · .gitignore

`.gitignore` lists files Git should not track: dependencies, build output, logs, local configuration and secrets. We write patterns, test them with `git check-ignore`, and fix the classic problem: a file that is already tracked is not affected by `.gitignore`.

VISUAL

```
.gitignore           matches
node_modules/       the folder node_modules anywhere (trailing / = only directories)
.env                 .env in any folder
*.log                app.log, logs/error.log
!keep.log            exception: do track keep.log
/build               build at the repository root only

untracked + matches .gitignore → ignored (not shown, not added by git add .)
ALREADY TRACKED               → .gitignore has NO effect: git rm --cached first
```

COMMANDS

Command / file	Use
<code>.gitignore</code> (any folder)	shared rules, committed
<code>.git/info/exclude</code>	personal rules for this clone, not committed
<code>git config --global core.excludesFile ~/.gitignore_global</code>	personal rules for every repository (<code>.DS_Store</code> , <code>.idea/</code>)
<code>git check-ignore -v PATH</code>	which file and line ignore PATH
<code>git status --ignored</code>	show ignored files too
<code>git rm --cached FILE</code>	stop tracking FILE, keep it on disk

RECAP

- `.gitignore` patterns keep dependencies, build output, logs and secrets out of Git.
- `git check-ignore -v` explains which rule applies.
- Tracked files are not affected: `git rm --cached`, commit, and rotate any secret that was committed.

Full lesson with every command and its real output: [73-gitignore](#)

Module 15 · Git internals

Lesson 74 · How Git stores data

Under every command you have used, Git stores only four kinds of objects in `.git/objects`, each named by the hash of its content: **blobs** (file contents), **trees** (folders), **commits** (snapshots with history) and **tags**. Plus **references**: small files with names like `main` pointing to commits.

VISUAL

```
refs/heads/main → commit 4267004      references: names → commit IDs
    tree
    parent fc345e6
    author, message
        tree 9a1c...      a folder: names → blob/tree IDs
        ├── README.md → blob 3f2e...  file content only (no name!)
        ├── menu.txt  → blob 7b04...
        └── prices.txt → blob c2a9...
```

same content = same hash = stored once. Change one byte → new blob → new tree → new commit.

COMMANDS

Command	What it does
<code>git cat-file -t ID</code>	the object's type: blob, tree, commit, tag
<code>git cat-file -p ID</code>	pretty-print the object
<code>git rev-parse REV:PATH</code>	the blob/tree ID of PATH in REV
<code>git hash-object FILE</code>	compute a file's blob ID (<code>-w</code> also stores it)
<code>git count-objects -v</code>	how many objects, how much space
<code>git ls-tree -r REV</code>	every file in a commit's snapshot

RECAP

- Objects: blob (content), tree (folder), commit (snapshot + parents + message), tag.
- Every object's name is the hash of its content; same content, same object.
- References (branches, tags, HEAD) are names pointing to commits.

Full lesson with every command and its real output: [74-how-git-stores-data](#)

Lesson 75 · Git objects

We build a commit **by hand** with plumbing commands (`hash-object` , `update-index` , `write-tree` , `commit-tree` , `update-ref`), to see that `git add` and `git commit` are nothing more than these steps. `git cat-file` lets us inspect every result.

VISUAL

```
git add menu.txt      = git hash-object -w menu.txt      (blob)
                     git update-index --add menu.txt    (staging area = the index)
git commit -m "msg"   = git write-tree                  (tree from the index)
                     git commit-tree TREE -p PARENT -m msg (commit object)
                     git update-ref refs/heads/main COMMIT (move the branch)
```

COMMANDS

Plumbing	Porcelain it is part of
<code>git hash-object -w FILE</code>	<code>git add</code> (store content)
<code>git update-index [--add] FILE</code>	<code>git add</code> (record in the index)
<code>git ls-files --stage</code>	what the index contains
<code>git write-tree</code>	<code>git commit</code> (snapshot the index)
<code>git commit-tree TREE -p PARENT</code>	<code>git commit</code> (create the commit)
<code>git update-ref REF ID</code>	<code>git commit</code> , <code>git branch -f</code> , <code>git reset</code> (move a reference)
<code>git cat-file -t / -s / -p ID</code>	inspect type / size / content

RECAP

- `git add` = store the blob + update the index; `git commit` = write a tree + create a commit + move the branch.
- `git cat-file` inspects any object.
- An object without a reference is invisible to `git log` but still in the store.

Full lesson with every command and its real output: [75-git-objects](#)

Lesson 76 · HEAD

HEAD is the answer to "where am I?". Normally it points to a **branch**, and the branch points to a commit. That indirection is why committing moves the branch you are on, and switching branches changes what **HEAD** points to.

VISUAL

```
.git/HEAD: ref: refs/heads/main
      |
      v
.git/refs/heads/main: 4267004...
      |
      v
      commit 4267004 "Add prices"

git commit      → new commit; main moves to it; HEAD still says "ref: refs/heads/main"
git switch tea   → HEAD becomes "ref: refs/heads/tea"
```

COMMANDS

Expression / command	Meaning
HEAD	the current commit (through the current branch)
HEAD~1 , HEAD~2	first parent, grandparent
HEAD^2	second parent (of a merge)
git symbolic-ref HEAD	which branch HEAD points to (fails if detached)
git branch --show-current	the same, friendlier
@	shorthand for HEAD

RECAP

- **HEAD** → branch → commit; committing moves the branch, switching moves **HEAD**.
- **HEAD~N** , **HEAD^2** , **@** navigate from there.
- **git symbolic-ref** reads and sets **HEAD** safely.

Full lesson with every command and its real output: [76-head](#)

Lesson 77 · Branches are references

A branch is not a copy of anything: it is a 41-byte file (or a line in `packed-refs`) containing one commit ID. Creating a branch is instant, deleting one deletes only that pointer, and a "branch's commits" are simply the commits reachable from it.

VISUAL

```
.git/refs/heads/main      → 4267004
.git/refs/heads/feature-tea → bb67674
.git/refs/tags/v1.0.0     → 4267004
.git/refs/remotes/origin/main → 4267004

main  —————> Commit C
      |
      |— parent B ...

feature-tea —————> Commit F
              |
              |— parent C ...
```

COMMANDS

Command	What it does
<code>git for-each-ref [refs/heads]</code>	list references with formats
<code>git show-ref</code>	references and IDs
<code>git update-ref REF ID</code>	create or move a reference (with a reflog entry)
<code>git update-ref -d REF</code>	delete a reference
<code>git pack-refs --all</code>	move loose reference files into <code>.git/packed-refs</code>
<code>git branch NAME ID</code>	the porcelain way to create a branch

RECAP

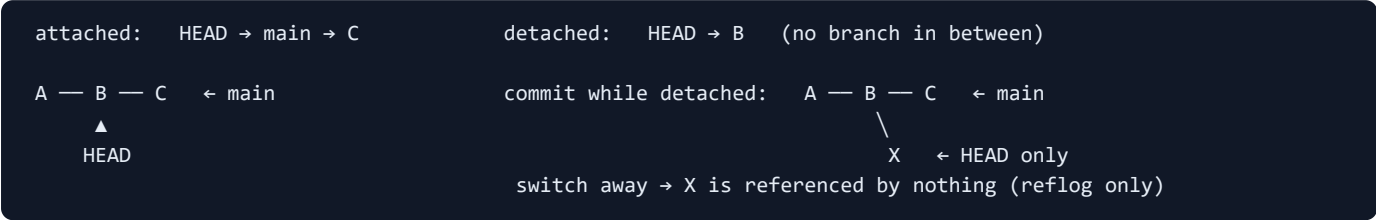
- A branch is a reference: a name containing a commit ID.
- Creating, moving and deleting branches only changes these pointers.
- Unreferenced commits stay in the object store until garbage collection.

Full lesson with every command and its real output: [77-branches-are-references](#)

Lesson 78 · Detached HEAD

"Detached HEAD" means `HEAD` points directly to a commit instead of to a branch. It happens when you check out a tag, a commit ID or a remote branch. Looking around is perfectly fine; commits made there belong to no branch and are easy to lose. We create the situation on purpose and recover safely.

VISUAL



COMMANDS

Command	Effect
<code>git switch --detach REV</code>	detach on purpose
<code>git checkout TAG / SHA</code>	detaches too (with a warning)
<code>git switch -c NAME</code>	create a branch here: no longer detached
<code>git switch BRANCH / git switch -</code>	go back to a branch
<code>git branch NAME SHA</code>	rescue a commit after leaving it

RECAP

- Detached HEAD = HEAD points at a commit, not a branch; normal when inspecting tags or CI checkouts.
- Commits made while detached belong to no branch: `git switch -c NAME` before leaving.
- Lost one anyway? `git reflog` + `git branch NAME SHA`.

Full lesson with every command and its real output: [78-detached-head](#)

Module 16 · Advanced recovery

Lesson 79 · Recover a deleted branch

Deleting a branch deletes a pointer; the commits remain until garbage collection removes unreachable objects (weeks, by default). We recover deleted branches three ways: from the reflog, from another clone, and with `git fsck` when the reflog has nothing.

VISUAL

```
git branch -D feature      refs/heads/feature deleted (and its own reflog with it)
                           commits: still in .git/objects, now unreachable

where to find them again:
1. git reflog              if HEAD was ever on the branch (you worked on it)
2. another clone / server  origin/feature in a teammate's clone, the PR on GitHub (refs/pull/N/head)
3. git fsck --unreachable  lists commits nothing points to (last resort, before gc)
```

COMMANDS

Situation	Recovery
you had the branch checked out	<code>git reflog</code> → <code>git branch NAME SHA</code>
it was pushed / a teammate has it	<code>git branch NAME origin/NAME</code> in their clone, push it back
a PR existed on GitHub	"Restore branch" button on the closed PR, or <code>git fetch origin pull/N/head:NAME</code>
none of the above	<code>git fsck --unreachable --no-reflogs</code> → inspect → <code>git branch NAME SHA</code>

RECAP

- A deleted branch is a deleted pointer; the commits stay until garbage collection.
- Reflog (if you were on it) → another clone or the PR → `git fsck --unreachable` as a last resort.
- `git branch NAME SHA` recreates it; push it back if it lived on the server.

Full lesson with every command and its real output: [79-recover-deleted-branch](#)

Lesson 80 · Recover a deleted commit

A commit can disappear from a branch in many ways: an interactive rebase that dropped it, an `--amend` that replaced it, a reset. We recover one with the reflog and `cherry-pick`, and then see the limit: once the reflog has expired and garbage collection has run, only another copy (the server, a colleague) can help.

VISUAL

```
A — B — C — D ← main      git rebase -i ... drop C      A — B — D' ← main
                                C, D: unreachable, still in the reflog
recovery:  git reflog → find C → git cherry-pick C      A — B — D' — C'

after git reflog expire --expire=now --all && git gc --prune=now:  C is gone from THIS repository
```

COMMANDS

Command	Use
<code>git reflog [--date=iso]</code>	every position of HEAD, with the action that caused it
<code>git log -g --grep-reflog=TEXT</code>	search the reflog
<code>git cherry-pick SHA</code>	re-apply a lost commit on the current branch
<code>git reset --hard HEAD@{N}</code>	jump the branch back to before the mistake (all of it)
<code>git fsck --unreachable</code>	unreachable objects, independent of the reflog

RECAP

- Dropped, amended or reset commits stay in the reflog: find them, cherry-pick or reset back.
- `reflog expire` + `gc --prune=now` make them unrecoverable locally.
- Then only other copies help: the server, colleagues, PR refs. Push your work.

Full lesson with every command and its real output: [80-recover-deleted-commit](#)

Lesson 81 · Recover after a hard reset

`git reset --hard` is the most feared command, but what it destroys depends on how far your work got: **committed** work is fully recoverable, **staged** work is recoverable with some digging (`git fsck --lost-found`), and only **unstaged** work is really gone. We recover each case carefully.

VISUAL

how far did the work get?	after git reset --hard	recovery
committed	commits unreachable	<code>git reflog / ORIG_HEAD</code> → reset or branch
staged (git add)	blobs dangling, no names	<code>git fsck --lost-found</code> → <code>.git/lost-</code>
found/other/ only edited in the folder	overwritten	none (editor history at best)

COMMANDS

Command	Use
<code>ORIG_HEAD</code>	where HEAD was before the last reset / merge / rebase
<code>git reflog</code>	every earlier position, with what moved it
<code>git branch rescue HEAD@{N}</code>	keep a position under a name before experimenting further
<code>git fsck --lost-found</code>	write dangling commits/blobs to <code>.git/lost-found/</code>
<code>git show BLOB</code>	read a recovered blob's content

RECAP

- Committed work survives `reset --hard` : `ORIG_HEAD` , `reflog`.
- Staged work survives as dangling blobs: `git fsck --lost-found` .
- Unstaged work does not survive: commit or stash before risky commands.

Full lesson with every command and its real output: [81-recover-after-hard-reset](#)

Module 17 · Git hooks

Lesson 82 · What are Git hooks?

Hooks are scripts Git runs automatically at certain moments: before a commit, after a checkout, before a push, when a server receives a push. They let you automate checks and tasks. We look at where they live, write a first one, and see the most common reason a hook "does nothing".

VISUAL

Git operation		hook (script in .git/hooks/, exact name, executable)	effect
git commit	→	pre-commit	exit ≠ 0 → commit aborted
		commit-msg	exit ≠ 0 → commit aborted
		post-commit	(informational)
git push	→	pre-push	exit ≠ 0 → push aborted
server receives	→	pre-receive	exit ≠ 0 → push rejected
			checks on the staged files
			checks on the message
			notifications, logs
			run tests before pushing
			branch protection (lesson 59)

COMMANDS

Command / path	Use
.git/hooks/NAME	a hook for this clone (exact name, no extension, executable, with a #! line)
git hook run NAME	run a hook manually to test it
git hook list NAME	which hook scripts Git would run
git config core.hooksPath DIR	use hooks from another directory (lesson 85)
git commit --no-verify	skip pre-commit and commit-msg (lesson 83)

RECAP

- Hooks are scripts in .git/hooks/ named after an event; non-zero exit stops pre-* operations.
- Exact name, executable, with a #! line; test with git hook run NAME .
- Hooks are local and not cloned: CI must enforce what really matters.

Full lesson with every command and its real output: [82-what-are-hooks](#)

Lesson 83 · Pre-commit hook

A practical `pre-commit` hook that checks the **staged** changes before every commit: it blocks AWS-style access keys, private keys and files over 1 MB. And why a local hook is a convenience, not a security control.

VISUAL

```
git commit
|
▼
.git/hooks/pre-commit  reads: git diff --cached (exactly what will be committed)
├─ AKIA... access key?      → exit 1: "commit blocked"
├─ -----BEGIN ... PRIVATE KEY → exit 1
├─ file > 1 MB?             → exit 1
└─ all fine                 → exit 0 → commit created
```

COMMANDS

Command	Use in a hook
<code>git diff --cached</code>	the staged changes (what will be committed)
<code>git diff --cached --name-only --diff-filter=AM</code>	added or modified staged files
<code>git hook run pre-commit</code>	test the hook without committing
<code>git commit --no-verify (-n)</code>	skip <code>pre-commit</code> and <code>commit-msg</code>
<code>gitleaks protect --staged / detect-secrets</code>	dedicated secret scanners to call from the hook

RECAP

- `pre-commit` checks `git diff --cached`; exit 1 aborts the commit.
- Block secrets, private keys and large files early, with clear messages.
- Local hooks can be skipped: enforce the same checks in CI and with push protection.

Full lesson with every command and its real output: [83-pre-commit-hook](#)

Lesson 84 · Commit message hook

The `commit-msg` hook receives the path of the file holding the commit message and can reject it. We enforce the **Conventional Commits** format (`feat: ...`, `fix(scope): ...`), which tools use to generate changelogs and version numbers.

VISUAL

```
git commit -m "fixed stuff"
|
▼
.git/hooks/commit-msg $1 = .git/COMMIT_EDITMSG (the message)
|   first line must match: type(optional-scope): description  (≤ 72 characters)
|   types: feat fix docs chore refactor test ci build perf
|   └─ no match → exit 1 → commit aborted, message explained
|   └─ match   → exit 0 → commit created
```

COMMANDS

Type	For
<code>feat</code>	a new feature (minor version in semantic-release)
<code>fix</code>	a bug fix (patch version)
<code>feat!</code> / <code>BREAKING CHANGE: footer</code>	an incompatible change (major version)
<code>docs</code> , <code>chore</code> , <code>refactor</code> , <code>test</code> , <code>ci</code> , <code>build</code> , <code>perf</code>	no release by themselves
<code>git commit --no-verify</code>	skips <code>commit-msg</code> too

RECAP

- `commit-msg` gets the message file as `$1`; exit 1 rejects the commit.
- Conventional Commits: `type(scope): description`, machine-readable history.
- Enforce it in CI as well; hooks can be skipped.

Full lesson with every command and its real output: [84-commit-msg-hook](#)

Lesson 85 · Git hooks in teams

Hooks in `.git/hooks/` belong to one clone: they are never committed, cloned or pushed. Teams share hooks by committing them to a normal folder and pointing Git at it with `core.hooksPath`, and they enforce the important checks in CI, because local hooks are optional by nature.

VISUAL

repository (committed)
 `.githooks/`
 ├─ `pre-commit`
 └─ `commit-msg`

each developer, once after cloning
`git config core.hooksPath .githooks`
 |
 ▼
 Git runs `.githooks/*` instead of `.git/hooks/*`

local hooks = fast feedback, skippable (`--no-verify`, not installed, other machine)
CI checks = the same checks on every PR, required by branch protection: not skippable

COMMANDS

Command / tool	Use
<code>git config core.hooksPath .githooks</code>	use the committed hooks folder (per clone)
<code>git config --global core.hooksPath ~/.githooks</code>	personal hooks for every repository
<code>git hook list commit-msg</code>	which hook Git would run
<code>make setup</code> / <code>npm prepare</code> / <code>pre-commit install</code>	a setup step that installs hooks automatically
CI job running the same checks	the real enforcement

RECAP

- Hooks are not cloned; share them as files in the repository plus `core.hooksPath`.
- Git never activates hooks by itself: a setup step is needed per clone.
- Local hooks give fast feedback; CI with branch protection gives enforcement.

Full lesson with every command and its real output: [85-hooks-in-teams](#)

Module 18 · Advanced repositories

Lesson 86 · Git submodules

A submodule embeds another Git repository inside yours, pinned to **one exact commit**. Useful for sharing a library, a chart or a set of templates between repositories; also a source of confusion, because cloning, updating and committing need extra steps. We go through all of them.

VISUAL

```
cafe (main repository)
├─ menu.txt
├─ .gitmodules          path = shared url = ../shared.git (committed)
└─ shared/ ───────────> shared repository, at commit 1a2b3c4 (the "gitlink": the main repo stores only
this ID)

main repo commit → tree → shared: commit 1a2b3c4    (not the files: a pointer to the other repo's commit)
```

COMMANDS

Command	What it does
<code>git submodule add URL PATH</code>	add a submodule, create/extend <code>.gitmodules</code>
<code>git clone --recurse-submodules URL</code>	clone including submodules
<code>git submodule update --init [--recursive]</code>	check out the pinned commits after a plain clone
<code>git submodule update --remote</code>	move submodules to their remote branch's latest commit
<code>git submodule status</code>	pinned commit of each submodule (<code>-</code> = not initialised)
<code>git config submodule.recurse true</code>	let <code>pull</code> / <code>switch</code> update submodules automatically

RECAP

- A submodule = another repository pinned to one commit (`.gitmodules` + a gitlink, mode 160000).
- Clone with `--recurse-submodules`, or `git submodule update --init` afterwards.
- Updating means: move the submodule, then commit the new pin in the main repository.

Full lesson with every command and its real output: [86-git-submodules](#)

Lesson 87 · Git worktrees

A worktree is an extra working directory attached to the same repository. Instead of stashing your half-finished feature to fix an urgent bug, you open the bug fix in a second folder, on another branch, at the same time. One repository, several checked-out branches.

VISUAL

```
lesson-87/      (main worktree)  branch feature-tea  ← half-finished work stays as it is
├── .git/        the ONE repository (objects, refs, config)
├── lesson-87-hotfix/ (linked worktree)  branch hotfix      ← urgent fix, in parallel
└── lesson-87-review/ (linked worktree)  detached at origin/... ← review a colleague's branch

rule: a branch can be checked out in only one worktree at a time
```

COMMANDS

Command	What it does
<code>git worktree add PATH BRANCH</code>	new working directory on an existing branch
<code>git worktree add -b NEW PATH START</code>	... on a new branch
<code>git worktree add --detach PATH REV</code>	... detached (review, builds)
<code>git worktree list</code>	all worktrees
<code>git worktree remove PATH</code>	delete a worktree (refuses if it has changes)
<code>git worktree prune</code>	forget worktrees whose folders were deleted manually

RECAP

- A worktree is another working directory of the same repository, on another branch.
- No stashing or second clone needed to work on two things at once.
- One branch per worktree; `git worktree remove` / `prune` to clean up.

Full lesson with every command and its real output: [87-git-worktrees](#)

Lesson 88 · Git LFS

Git keeps every version of every file forever, in every clone. For large binaries (images, videos, datasets, model files, installers) that makes repositories huge and slow. **Git LFS** (Large File Storage) stores those files on a separate server and keeps only small pointer files in Git. We set it up, look at a pointer, and fix a big file that was committed without LFS.

VISUAL

without LFS: every version of logo.psd (50 MB × 30 versions) inside .git → every clone downloads 1.5 GB

with LFS: Git stores a 130-byte pointer: LFS server stores the content:
version https://git-lfs.github.com/spec/v1 oid sha256:4d7a... → 50 MB
oid sha256:4d7a...
size 52428800
clone/checkout downloads only the versions actually checked out

GitHub rejects files over 100 MB (GH001: Large files detected) and warns from 50 MB.

COMMANDS

Command	What it does
<code>git lfs install</code>	enable LFS for your user (once)
<code>git lfs track "PATTERN"</code>	manage matching files with LFS (writes <code>.gitattributes</code>)
<code>git lfs ls-files</code>	files in LFS at HEAD
<code>git lfs pull</code>	download LFS content for the current checkout
<code>GIT_LFS_SKIP_SMUDGE=1 git clone ...</code>	clone with pointers only (fast CI)
<code>git lfs migrate import --include="PATTERN"</code>	rewrite history: move existing files into LFS

RECAP

- LFS keeps large binaries out of Git history; Git stores pointers.
- `git lfs track` before adding the files; commit `.gitattributes` .
- Large files already committed: `git lfs migrate import` (rewrites history).

Full lesson with every command and its real output: [88-git-lfs](#)

Module 19 · Git security

Lesson 89 · Secrets in Git

Why a password, API token or access key in a Git repository is a serious incident: Git remembers every version, every clone has a full copy, and deleting the file in a new commit does **not** remove it. We commit a (fake) secret, try to "delete" it, find it again, and set up the habits that prevent it.

VISUAL

commit 1 add .env (DB_PASSWORD=..., API_TOKEN=...)

commit 2 git rm .env "remove secrets"

← the secret is now in a blob

← the latest snapshot has no .env ...

... but commit 1 still has it, in every clone,

in forks, CI caches, and on GitHub

Rule: a secret that reached a shared repository is COMPROMISED → revoke/rotate it first, clean history second.

Typical secrets: `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` , `PASSWORD` , `API_TOKEN` , private keys (`id_rsa` , `*.pem`), `.env` files, kubeconfigs, Terraform state.

COMMANDS

Command	Finds
<code>git log --all -- FILE</code>	every commit that touched FILE, even deleted
<code>git log -p -S "TEXT" --all</code>	commits that added or removed TEXT
<code>git grep "PATTERN" \$(git rev-list --all)</code>	PATTERN in every version of every file
<code>gitleaks detect</code> / <code>trufflehog git file://.</code>	dedicated scanners for many secret formats, whole history

RECAP

- Deleting a secret in a new commit does not remove it from history or from clones.
- A pushed secret is compromised: rotate it first, then clean history.
- Prevent: `.gitignore` , `.env.example` , secret managers, pre-commit and server-side scanning.

Full lesson with every command and its real output: [89-secrets-in-git](#)

Lesson 90 · Removing sensitive data

How to rewrite a repository's history so that a file (or a string) never existed, with `git filter-repo`, and what else has to happen for the cleanup to be real: force-pushing, every clone being replaced, and server-side caches. The credential itself must already be rotated (lesson 89): history rewriting is cleanup, not protection.

VISUAL

```
before:  A — B(+.env) — C — D(-.env) — E      every commit after B has a new parent ...
after:   A — B' ————— C' — D' ————— E'    ... so B..E all get NEW IDs; .env exists in none of them

then:    git push --force (all branches and tags) → server has only the new history
         every teammate RE-CLONES (an old clone pushed again brings the secret back)
         GitHub: cached views / PR refs may still show old commits → GitHub Support can purge them
```

COMMANDS

Command	Removes
<code>git filter-repo --invert-paths --path FILE</code>	FILE from every commit
<code>git filter-repo --invert-paths --path-glob '*.pem'</code>	all matching files
<code>git filter-repo --replace-text replacements.txt</code>	strings (<code>OLD==>***REMOVED***</code> lines) in all files
<code>git filter-repo --analyze</code>	report of large/deleted files, to decide what to remove
BFG Repo-Cleaner	an older alternative (Java)
<code>git filter-branch</code>	the built-in, slow and error-prone predecessor: avoid

RECAP

- Rotate first; rewrite history second.
- `git filter-repo --invert-paths --path FILE` (or `--replace-text`) on a mirror clone, then force-push everything.
- Every clone must be replaced; otherwise the old history comes back with the next push.

Full lesson with every command and its real output: [90-removing-sensitive-data](#)

Lesson 91 · Commit signing

Anyone can write any name and email into a commit (lesson 04: it is just configuration). A **signature** proves that the commit was made by the holder of a private key. We sign commits and tags with an SSH key (simplest, Git 2.34+), verify them, and see how GPG does the same.

VISUAL

```
git commit -S      commit object + signature made with YOUR private key
                    |
git verify-commit  checks the signature with the PUBLIC key:
                    SSH:  allowed_signers file ("ada@example.com ssh-ed25519 AAAA...")
                    GPG:  your keyring
                    GitHub: the public key uploaded as a "signing key" → "Verified" badge

author name/email = a claim · valid signature from a trusted key = proof
```

COMMANDS

Command / setting	Use
<code>git config gpg.format ssh</code>	sign with SSH keys (default: <code>openpgp</code> = GPG)
<code>git config user.signingkey KEY</code>	the key (SSH: the <code>.pub</code> path; GPG: the key ID)
<code>git config commit.gpgsign true</code> / <code>tag.gpgsign true</code>	sign every commit / tag automatically
<code>git commit -S</code> , <code>git tag -s</code>	sign one commit / tag
<code>git verify-commit C</code> , <code>git verify-tag T</code>	verify
<code>git log --show-signature</code> / <code>--format='%G? %h %s'</code>	signatures in the log (<code>G</code> good, <code>N</code> none, <code>B</code> bad, <code>U</code> unknown key)

RECAP

- Author fields are claims; signatures are proof of key possession.
- SSH signing: `gpg.format ssh`, `user.signingkey`, `commit.gpgsign`; verify with an allowed signers file.
- Upload the signing key to GitHub for "Verified", and require signed commits on protected branches.

Full lesson with every command and its real output: [91-commit-signing](#)

Lesson 92 · Supply chain security

Your software is built from repositories you do not control: libraries, Helm charts, GitHub Actions, base images. The Git-level risks: **mutable references** (a tag or branch can be moved to malicious code), **spoofed authors**, **compromised accounts** and **malicious commits** hidden in large changes. We exploit a moved tag locally, then pin by commit ID.

VISUAL

```
your workflow:  uses: some-org/deploy-action@v2           ← a tag: the owner (or an attacker with their token)
                  can move it to any commit, any time
                  uses: some-org/deploy-action@3f9a1c...    ← a full commit ID: immutable content

trust signals:  signed commits/tags from known keys · protected branches with reviews · trusted publishers ·
                Dependabot/Renovate updating pins through PRs · least-privilege tokens in CI
```

COMMANDS

Practice	Commands / settings
pin dependencies to commit IDs	<code>uses: org/action@<40-char sha></code> , <code>git rev-parse v1.0.0^{commit}</code>
resolve a tag before pinning	<code>git ls-remote https://github.com/actions/checkout refs/tags/v4</code>
verify signed tags/commits	<code>git verify-tag</code> , <code>git log --format='%G? %h %an'</code>
detect moved tags	compare stored IDs; Git refuses to overwrite existing local tags on fetch
least privilege in CI	<code>permissions: contents: read</code> in workflows, fine-grained tokens
updates through review	Dependabot / Renovate PRs that bump the pinned SHA

RECAP

- Tags and branches are mutable; commit IDs are not: pin dependencies by commit ID.
- Author names are claims; trust signatures from known keys, reviews and protected branches.
- Least-privilege tokens and 2FA limit what a compromised account or action can do.

Full lesson with every command and its real output: [92-supply-chain-security](#)

Module 20 · GitHub advanced

Lesson 93 · GitHub Issues

Issues track work: bugs, feature requests, tasks. The useful part for Git users is the link between issues and commits: writing `Fixes #12` in a commit or PR closes the issue automatically when the change reaches the default branch. We run that workflow on the practice repository.

VISUAL

```
Issue #N "Green tea has no price" (open)
  | assigned, labelled (lesson 94), in a milestone (lesson 95)
  ▼
branch fix-green-tea-price → commit "Price green tea (fixes #N)" → PR → merge into main
  |
  ▼
Issue #N closed automatically, with a link to the commit / PR
closing keywords: close(s/d), fix(es/ed), resolve(s/d) + #N (only on the DEFAULT branch)
```

COMMANDS

Command	What it does
<code>gh issue create --title T --body B [--label L --assignee @me]</code>	open an issue
<code>gh issue list [--state all] [--label L] [--search Q]</code>	list / filter (<code>--search</code> uses GitHub search, which can lag a little)
<code>gh issue view N [--comments]</code>	details
<code>gh issue comment N --body B</code>	comment
<code>gh issue close N [--reason "not planned"] / reopen</code>	close / reopen by hand
<code>Fixes #N</code> in a commit or PR description	close N when merged into the default branch

RECAP

- Issues track work; `gh issue create/list/view/comment/close`.
- `Fixes #N` / `Closes #N` in a commit or PR closes the issue when it reaches the default branch.
- On other branches the commit is only referenced; merging closes it.

Full lesson with every command and its real output: [93-github-issues](#)

Lesson 94 • Labels

Labels categorise issues and pull requests: type (`bug` , `enhancement`), priority, area (`helm` , `terraform`), status (`needs-review`). They make lists filterable and drive automation (a `release-notes` label, a CI job that runs only for `deploy-preview`). We create labels, apply them and filter by them.

VISUAL

```
#31 Espresso machine shows wrong price  [bug] [priority:high] [area:prices]
#32 Add chai                             [enhancement] [area:menu]
#33 Update the Helm chart                 [area:helm] [needs-review]

gh issue list --label "priority:high" → #31
```

COMMANDS

Command	What it does
<code>gh label list</code>	labels of the repository
<code>gh label create NAME --color HEX --description D [--force]</code>	create (or update with <code>--force</code>)
<code>gh label edit NAME --name NEW</code> / <code>gh label delete NAME --yes</code>	rename / delete
<code>gh issue edit N --add-label L --remove-label L</code>	change an issue's labels (also <code>gh pr edit</code>)
<code>gh issue list --label L</code>	filter (several <code>--label</code> = all of them)
<code>gh label clone OTHER/REPO</code>	copy a label set from another repository

RECAP

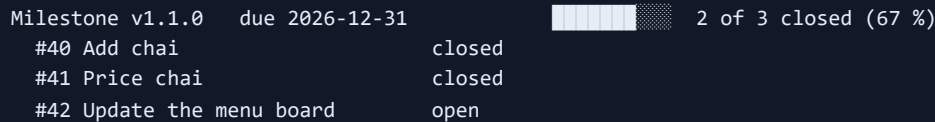
- Labels categorise issues and PRs by type, priority, area, status.
- `gh label create/list/edit/delete` ; apply with `gh issue edit --add-label` .
- Labels must exist before use; share one scheme across repositories.

Full lesson with every command and its real output: [94-labels](#)

Lesson 95 • Milestones

A milestone groups issues and pull requests toward a goal with an optional due date, typically a release ("v1.1.0"). GitHub shows its progress as the share of closed items. We create one, assign issues, close them, and read the progress.

VISUAL



COMMANDS

Command	What it does
<code>gh api repos/O/R/milestones -f title=T -f due_on=ISO</code>	create
<code>gh api repos/O/R/milestones --jq ...</code>	list with progress (<code>open_issues</code> , <code>closed_issues</code>)
<code>gh issue create ... --milestone T / gh issue edit N --milestone T</code>	assign
<code>gh issue list --milestone T</code>	the milestone's issues
<code>gh api -X PATCH repos/O/R/milestones/N -f state=closed</code>	close the milestone

RECAP

- A milestone groups issues/PRs toward a goal, usually a release, with progress and a due date.
- Create and inspect with `gh api .../milestones` ; assign with `--milestone TITLE` .
- Titles must match exactly.

Full lesson with every command and its real output: [95-milestones](#)

Lesson 96 · Projects

GitHub Projects are planning boards and tables built on top of issues and pull requests, across one or many repositories: columns such as Todo / In progress / Done, custom fields (priority, estimate, iteration), and views. They do not change Git at all; they organise the work around it.

VISUAL

Project "Cafe roadmap" (owned by you or an organisation, can span repositories)

Todo	In progress	Done
#42 Update the menu board priority: high · v1.1.0	#45 Seasonal drinks @ada	#40 Add chai #41 Price chai

items = issues / PRs (or draft notes) · Status, Priority ... = project fields · board / table / roadmap = views

COMMANDS

Command	What it does
<code>gh auth refresh -s project</code>	give the CLI token access to Projects (browser confirmation)
<code>gh project create --owner OWNER --title T</code>	create a project
<code>gh project list --owner OWNER</code>	list projects
<code>gh project item-add N --owner OWNER --url ISSUE_URL</code>	add an issue/PR
<code>gh project item-list N --owner OWNER</code>	items with their fields
<code>gh project field-list N --owner OWNER</code>	fields (Status, custom ones)
<code>gh project link N --owner OWNER --repo REPO</code>	show the project in a repository's Projects tab

RECAP

- Projects organise issues and PRs into boards, tables and roadmaps, across repositories.
- The CLI needs the `project` (or `read:project`) scope: `gh auth refresh -s project`.
- Project workflows keep the board in sync with issue and PR states automatically.

Full lesson with every command and its real output: [96-projects](#)

Lesson 97 · Releases

A GitHub Release is a tag (lesson 68) plus a page: title, release notes, and downloadable files (binaries, archives, checksums). We publish three versions of the cafe menu, **v1.0.0**, **v1.1.0** and **v2.0.0**, following semantic versioning, with generated notes and an attached asset.

VISUAL

```
semantic versioning  MAJOR.MINOR.PATCH
v1.0.0  first release
v1.1.0  new things, nothing broken      (feat)
v2.0.0  something incompatible changed  (feat! / BREAKING CHANGE)
v2.0.1  bug fix only                   (fix)

gh release create v1.1.0 --target <commit> --title ... --generate-notes menu.tar.gz
└─ tag v1.1.0 (created if missing) + release page + notes from merged PRs + asset
```

COMMANDS

Command	What it does
gh release create TAG [FILES] --target REV --title T -notes N	publish (creates the tag if missing)
--generate-notes	notes from merged PRs and contributors since the previous release
--prerelease / --draft / --latest=false	mark as pre-release / keep unpublished / not "Latest"
gh release list / view TAG	releases / one release
gh release upload TAG FILE / download TAG	add / fetch assets
gh release edit TAG --notes ...	change a release
gh release delete TAG --cleanup-tag	delete it and its tag

RECAP

- Release = tag + notes + assets; versions follow MAJOR.MINOR.PATCH.
- gh release create/list/view/edit/delete ; --generate-notes for automatic notes.
- Never reuse a version for different content: edit notes, release new versions.

Full lesson with every command and its real output: [97-releases](#)

Lesson 98 · GitHub Actions introduction

How Git events start automation: a push or a pull request triggers a **workflow** defined in the repository (`.github/workflows/*.yaml`), and GitHub runs it on a fresh machine. This lesson is the bridge from Git to CI/CD: one small workflow, a failing run, and the fix. GitHub Actions itself (jobs, matrices, secrets, OIDC, deployments) is a course of its own.

VISUAL

```
git push → GitHub → event "push" on main → workflow .github/workflows/check.yml
                                     runs-on: ubuntu-latest
                                     steps: checkout → check the price file
                                             ✓ green / X red   (shown on the commit and on PRs)

branch protection (lesson 59) can REQUIRE the check before merging
```

COMMANDS

Command / file	What it does
<code>.github/workflows/NAME.yaml</code>	a workflow: <code>on:</code> (events), <code>jobs:</code> → <code>steps:</code>
<code>gh workflow list</code> / <code>gh workflow run NAME</code>	list / start one manually (with <code>workflow_dispatch</code>)
<code>gh run list [--workflow W]</code>	recent runs
<code>gh run watch ID --exit-status</code>	follow a run, exit non-zero if it fails
<code>gh run view ID [--log-failed]</code>	result / logs of the failed steps
<code>gh pr checks</code>	the checks of a pull request

RECAP

- Workflows in `.github/workflows/` run on Git events (push, pull_request, tags, manual).
- `gh run list` / `watch` / `view --log-failed` to follow and debug runs.
- Required checks plus branch protection turn CI into a gate for `main` .

Full lesson with every command and its real output: [98-github-actions-intro](#)

Module 21 · Professional DevOps Git workflow

Level 21 · DevOps workflow · ⌚ 60 minutes · run every command from the course folder (`git-practical-course/`)

What are we learning?

Everything from the previous modules, in the order a DevOps team uses it every day, on a realistic repository: an application, a Dockerfile, a Helm chart, Kubernetes manifests, scripts, documentation and CI workflows. One change goes from a developer's branch all the way to production.

Visual

```
Developer → Feature branch → Commit → Push → Pull Request → Review → CI → Merge → Release →  
Deployment  
Ada      feat/add-chai    feat(...)  origin    branch vs main  Grace    ci.sh  --no-ff  v1.1.0  
production/  
  
(deploy.sh)                                (tag)
```

The repository (`devops-project/`):

```
devops-project/  
├─ application/      index.html, menu.json          the website  
├─ Dockerfile        nginx (non-root) serving application/  
├─ VERSION           1.0.0: the single source of truth for the version  
├─ helm/cafe/        Chart.yaml (appVersion), values.yaml (image tag), templates/  
├─ kubernetes/       namespace.yaml  
├─ scripts/          ci.sh · release.sh · deploy.sh  
├─ docs/             CONTRIBUTING.md (the team's workflow)  
└─ .github/workflows/ ci.yml (PRs, main) · release.yml (tags v*: image → GHCR, GitHub release)
```

On GitHub the pull request, review and CI happen on github.com (modules 10–12, 20); here the "server" is a local bare repository and the CI is `scripts/ci.sh`, the same script the GitHub workflow runs. The capstone (module 24) does the whole flow on GitHub.

Lab setup

Create the team's repository from the template, publish it on the "server", and give Grace (the reviewer) a clone:

```
bash scripts/new-lab.sh devops empty  
cp -r 21-devops-workflow/devops-project/. ~/git-practice/devops/  
cd ~/git-practice/devops  
git add . && git commit -q -m "chore: initial cafe DevOps repository"  
git tag -a v1.0.0 -m "Release 1.0.0"  
git init -q --bare ../devops-server.git && git remote add origin ../devops-server.git  
git push -q -u origin main --follow-tags  
git clone -q ../devops-server.git ../devops-grace  
git -C ../devops-grace config user.name "Grace Hopper" && git -C ../devops-grace config user.email  
"grace@example.com"  
git ls-remote --heads --tags ../devops-server.git | sed 's|.*\trefs/|' && echo "../devops-server.git  
ready"
```

The current release is live:

```
bash scripts/deploy.sh ../devops-server.git v1.0.0 ../production
```

```
deployed cafe 1.0.0 (v1.0.0) to production
```

Demonstration

1. Developer → feature branch → commit

Ada adds chai to the menu, on a branch named by convention, with a Conventional Commit message:

```
git switch -q main && git pull -q
git switch -c feat/add-chai
sed -i 's/{"name": "cappuccino", "price": "3.40"}/{"name": "cappuccino", "price": "3.40"},\n    {"name": "chai", "price": "3.10"}/' application/menu.json
git diff --stat
git commit -q -am "feat(menu): add chai"
git log --oneline -1
```

2. CI locally, then push

```
bash scripts/ci.sh
git push -q -u origin feat/add-chai
```

```
ok    menu.json is valid JSON
ok    every menu item has a price with two decimals
ok    VERSION is semantic (MAJOR.MINOR.PATCH)
ok    Chart appVersion matches VERSION (1.0.0)
ok    image tag matches VERSION (1.0.0)
ok    no secrets in tracked files
all checks passed
```

3. Pull request → review → CI

The PR is "merge `feat/add-chai` into `main`". Grace reviews exactly what GitHub would show (lesson 51) and runs the checks on the branch (what `ci.yml` does on every PR):

```
cd ../devops-grace && git fetch -q
git log --oneline main..origin/feat/add-chai
git diff main...origin/feat/add-chai
git switch -q --detach origin/feat/add-chai && bash scripts/ci.sh | tail -1 && git switch -q main
```

```
a5addca (origin/feat/add-chai) feat(menu): add chai
diff --git a/application/menu.json b/application/menu.json
index 6ed58dd..e10981d 100644
--- a/application/menu.json
+++ b/application/menu.json
@@ -2,6 +2,7 @@
  "items": [
    {"name": "espresso", "price": "2.50"},
    {"name": "latte", "price": "3.20"},
-   {"name": "cappuccino", "price": "3.40"}
+   {"name": "cappuccino", "price": "3.40"},
+   {"name": "chai", "price": "3.10"}
  ]
}
```

all checks passed

4. Merge

Approved and green: merged with a merge commit (the PR button), and the branch is deleted:

```
git merge -q --no-ff -m "Merge pull request #1 from feat/add-chai" origin/feat/add-chai
git push -q && git push -q origin --delete feat/add-chai
git log --oneline --graph -4
```

```
* 92e6a18 (HEAD -> main, origin/main, origin/HEAD) Merge pull request #1 from feat/add-chai
|\
| * a5addca feat(menu): add chai
|/
* c12c5f4 (tag: v1.0.0) chore: initial cafe DevOps repository
```

5. Release

Ada updates `main` and releases 1.1.0 (a new feature: minor version). The script sets the version in `VERSION`, the chart and the image tag, re-runs the checks, commits and tags:

```
cd ../devops && git switch -q main && git pull -q && git branch -d feat/add-chai
bash scripts/release.sh 1.1.0
git push -q --follow-tags
git log --oneline --decorate -2
```

```
Deleted branch feat/add-chai (was a5addca).
released v1.1.0: push with git push --follow-tags
a594614 (HEAD -> main, tag: v1.1.0, origin/main, origin/HEAD) chore(release): 1.1.0
92e6a18 Merge pull request #1 from feat/add-chai
```

On GitHub, pushing the tag `v1.1.0` starts `release.yml`: image `ghcr.io/.../cafe:1.1.0` and a GitHub release with the packaged chart.

6. Deployment

```
bash scripts/deploy.sh ../devops-server.git v1.1.0 ../production
grep -c '"name"' ../production/application/menu.json
```

```
deployed cafe 1.1.0 (v1.1.0) to production
4
```

Production serves four items; the deployed version is exactly the tagged commit.

Command breakdown

Step	Commands	Lesson
start	<code>git switch main && git pull && git switch -c feat/...</code>	20, 41, 56
commit	<code>git add</code> , <code>git commit -m "feat(scope): ..."</code>	09–11, 84
check + push	<code>bash scripts/ci.sh</code> , <code>git push -u origin feat/...</code>	42–43
review	<code>git log main...BRANCH</code> , <code>git diff main...BRANCH</code> , <code>gh pr review</code>	51–53

Step	Commands	Lesson
merge	PR merge button / <code>git merge --no-ff</code>	23–25, 54
release	<code>scripts/release.sh X.Y.Z</code> → annotated tag, <code>git push --follow-tags</code>	68–69, 97
deploy	<code>deploy.sh</code> (real life: <code>docker build/push</code> , <code>helm upgrade --install --set image.tag=X.Y.Z</code>)	98

Hands-on exercise

Instructions. Show which commits went into release 1.1.0 since 1.0.0, without merges: the material for its release notes.

Expected result. "feat(menu): add chai" and "chore(release): 1.1.0".

Verification.

```
cd ~/git-practice/devops
git log --oneline --no-merges v1.0.0..v1.1.0
```

Break it

The next change: Ada raises the latte price and, "while she is at it", bumps the image tag by hand. She skips the local checks and pushes the branch:

```
git switch -q -c fix/latte-price
sed -i 's/"latte", "price": "3.20"/"latte", "price": "3.30"/' application/menu.json
sed -i 's/tag: "1.1.0"/tag: "1.1.1"/' helm/cafe/values.yaml
git commit -q -am "fix(menu): latte costs 3.30" && git push -q -u origin fix/latte-price
cd ../devops-grace && git fetch -q && git switch -q --detach origin/fix/latte-price
bash scripts/ci.sh
```

```
ok    menu.json is valid JSON
ok    every menu item has a price with two decimals
ok    VERSION is semantic (MAJOR.MINOR.PATCH)
ok    Chart appVersion matches VERSION (1.1.0)
FAIL  image tag matches VERSION (1.1.0)
ok    no secrets in tracked files
checks failed
```

Troubleshoot

The PR's CI is red: `image tag matches VERSION` failed. `values.yaml` now says `1.1.1` while `VERSION` and the chart say `1.1.0`: the deployment would run an image that was never built. In review, Grace points at the line:

```
git diff origin/main...origin/fix/latte-price -- helm/
git switch -q main
```

```
diff --git a/helm/cafe/values.yaml b/helm/cafe/values.yaml
index 1b84e81..b6a69a1 100644
--- a/helm/cafe/values.yaml
+++ b/helm/cafe/values.yaml
@@ -2,7 +2,7 @@ replicaCount: 2

image:
  repository: ghcr.io/your-account/cafe
- tag: "1.1.0"
+ tag: "1.1.1"

# for a private registry: a docker-registry secret, e.g. [{name: ghcr}]
imagePullSecrets: []
```

Versions are changed **only** by the release script, at release time; a PR changes the application.

Fix

Ada removes the version change from her commit (her own branch: amend and force-push with lease, lesson 42), CI goes green, and the PR can be merged and released as 1.1.1:

```
cd ../devops
git checkout main -- helm/cafe/values.yaml
git commit -q --amend --no-edit && bash scripts/ci.sh | tail -1
git push -q --force-with-lease
cd ../devops-grace && git fetch -q && git merge -q --no-ff -m "Merge pull request #2 from fix/latte-price"
origin/fix/latte-price && git push -q && git push -q origin --delete fix/latte-price
cd ../devops && git switch -q main && git pull -q && git branch -D fix/latte-price > /dev/null
bash scripts/release.sh 1.1.1 > /dev/null && git push -q --follow-tags
bash scripts/deploy.sh ../devops-server.git v1.1.1 ../production
```

```
all checks passed
deployed cafe 1.1.1 (v1.1.1) to production
```

Real-world example

This is the shape of most production repositories: protected `main` (lesson 59) requiring the `ci` check and one review; Conventional Commits (lesson 84) so release notes write themselves; tags trigger the release pipeline (image to a registry, chart to a repository); a deployment tool (Helm in CI, or Argo CD/Flux watching the Git repository: GitOps) makes the cluster follow what Git says. Rollback = deploy the previous tag.

Practice challenge

Production has a problem with 1.1.1. Roll back to 1.1.0 using only the release tags.

▼ Solution

```
cd ~/git-practice/devops
git tag -l "v*"
bash scripts/deploy.sh ../devops-server.git v1.1.0 ../production
grep latte ../production/application/menu.json
```

```
v1.0.0
v1.1.0
v1.1.1
deployed cafe 1.1.0 (v1.1.0) to production
{"name": "latte", "price": "3.20"},
```

Because every release is an immutable tag, rollback is a redeploy of an earlier tag; the fix then goes forward as 1.1.2.

Recap

- The DevOps flow: branch → commit → push → PR → review → CI → merge → release (tag) → deploy.
- CI scripts in the repository run the same locally and in GitHub Actions.
- Versions change only at release; releases are tags; deployment and rollback use tags.

Cleanup

```
cd ~ && rm -rf ~/git-practice/devops ~/git-practice/devops-server.git ~/git-practice/devops-grace ~/git-practice/production
```

Module 22 · Real-world troubleshooting

Level 22 · 18 hands-on problems · ⌚ 15–20 minutes each · run every command from the course folder

Each problem creates a broken situation on purpose and walks through it the way an experienced engineer would:

Problem → Symptoms → Investigation → Commands → Understand the output → Root cause → Fix → Verification → Prevention

Do them in any order. Each one creates its own lab in `~/git-practice/lesson-tNN` and removes it at the end.

#	Problem	Typical message	Related lessons
1	Committed on the wrong branch	(none: commits are just in the wrong place)	17, 21, 31
2	Merge conflict	CONFLICT (content): Merge conflict in ...	26–28
3	Accidental commit	(a file that does not belong in the commit)	30, 31, 73
4	Accidental reset	(commits missing from <code>git log</code>)	31, 33, 81
5	Deleted branch	fatal: invalid reference: ...	22, 79
6	Detached HEAD	Warning: you are leaving 1 commit behind	76, 78
7	Wrong remote	refusing to merge unrelated histories	37, 38
8	Push rejected	! [rejected] main -> main (fetch first)	40–42
9	Non-fast-forward error	! [rejected] ... (non-fast-forward)	42, 60, 63
10	Authentication failure	Permission denied (publickey)	48–50
11	Wrong upstream branch	The upstream branch of your current branch does not match ...	43
12	Accidentally committed secret	(a scanner or secret-scanning alert)	83, 89, 90
13	Large file rejected	GH001: Large files detected	83, 88
14	Rebase conflict	could not apply ... / interactive rebase in progress	64–66
15	Incorrect merge	(the wrong branch is in <code>main</code>)	23, 32, 54
16	Lost commit	(a commit vanished after a rebase)	33, 80
17	Wrong commit needs to be removed	(a bad commit in the middle of history)	31, 32, 63
18	Production branch contains an unwanted commit	(production behaves like the next release)	32, 58, 67

The general method

1. **Stop and look** before typing more commands: `git status`, `git log --oneline --graph --all -10`.
2. **Is it pushed?** `git branch -r --contains <sha>` / `git status` (ahead/behind). Unpushed = rewrite freely; pushed = add commits (revert) instead of rewriting.
3. **Find the last good state:** `git reflog`, `ORIG_HEAD`, the server, a teammate's clone.

4. **Mark it** before experimenting: `git branch rescue <sha>`.
5. **Fix, then verify** with the same commands that showed the problem.
6. **Prevent**: a hook, a protection rule, a habit.

Git command reference lab

Level 23 · organised by purpose · every command links to the lesson that teaches it · run the practice blocks from the course folder (`git-practical-course/`)

This is not a list to memorise. For each purpose: the commands, when to reach for them, the lesson where you used them, and a short practice block that runs them on a fresh lab so you can see the output again.

Lab setup

One small lab per section (the same scenarios as in the lessons):

```
bash scripts/new-lab.sh ref-changes basic
bash scripts/new-lab.sh ref-branches diverged
bash scripts/new-lab.sh ref-remote remote
bash scripts/new-lab.sh ref-recovery history
bash scripts/new-lab.sh ref-advanced bisect
bash scripts/new-lab.sh ref-internals basic
```

1 · Repository

Command	Use it when	Lesson
<code>git init [-b main]</code>	starting a new project	06
<code>git clone URL [DIR]</code>	getting an existing project	39
<code>git clone --depth 1 / --branch B</code>	fast CI clones / a specific branch	39
<code>git config --global user.name/email</code>	once per computer	04
<code>git config --list --show-origin</code>	"where does this setting come from?"	04

Practise:

```
mkdir -p ~/git-practice/ref-repo && cd ~/git-practice/ref-repo
git init -b main
git config --show-origin --get user.name || echo "user.name not configured yet (lesson 04)"
```

```
Initialized empty Git repository in ~/git-practice/ref-repo/.git/
file:~/.gitconfig  Ada Lovelace
```

2 · Changes

Command	Use it when	Lesson
<code>git status [--short]</code>	always: before and after everything	08
<code>git add FILE</code> / <code>git add -p</code>	choosing what goes into the next commit	09, 10
<code>git diff</code> / <code>git diff --cached</code>	unstaged / staged changes	15
<code>git restore FILE</code>	throwing away unstaged edits (permanent)	29
<code>git restore --staged FILE</code>	unstaging (safe)	30
<code>git rm [--cached] FILE</code> / <code>git mv</code>	deleting / untracking / renaming	73

Practise:

```
cd ~/git-practice/ref-changes
echo "chai" >> menu.txt && sed -i 's/latte 3.20/latte 3.30/' prices.txt
git add menu.txt
git status --short
git diff --cached --stat
```

```
M menu.txt
M prices.txt
menu.txt | 1 +
1 file changed, 1 insertion(+)
```

3 · Commits

Command	Use it when	Lesson
<code>git commit -m "type(scope): message"</code>	recording a snapshot	11, 84
<code>git commit --amend</code>	fixing the last (unpushed) commit	12
<code>git log --oneline --graph --all</code>	seeing the history	13, 16
<code>git log -S TEXT / -L 2,2:FILE / --author</code>	finding when something changed	13, 71
<code>git show COMMIT[:FILE]</code>	one commit / a file at a commit	14
<code>git blame -L FILE</code>	why is this line like this?	71

Practise:

```
cd ~/git-practice/ref-changes
git commit -q -m "feat(menu): add chai"
git log --oneline --graph -3
git show --stat --format='%h %an %s' HEAD
```

```
* e798c1c (HEAD -> main) feat(menu): add chai
* 4267004 Add prices
* fc345e6 Add the menu
e798c1c Ada Lovelace feat(menu): add chai

menu.txt | 1 +
1 file changed, 1 insertion(+)
```

4 · Branches

Command	Use it when	Lesson
<code>git switch -c NAME / git switch NAME</code>	starting / changing tasks	18–20
<code>git branch [-vv] [-d/-D NAME]</code>	listing / deleting	21, 22
<code>git merge [--no-ff/--ff-only] BRANCH</code>	integrating a branch	23–28
<code>git rebase main / git rebase -i BASE</code>	updating / cleaning your own branch	60–66
<code>git merge --abort / git rebase --abort</code>	backing out	28, 65

Practise:

```
cd ~/git-practice/ref-branches
git branch -vv
git merge -q --no-edit feature-tea
git log --oneline --graph -4
```

```
feature-tea bb67674 Add green tea to the menu
* main          f40d080 Add opening hours
* 38c0668 (HEAD -> main) Merge branch 'feature-tea'
|\
| * bb67674 (feature-tea) Add green tea to the menu
* | f40d080 Add opening hours
|/
* 4267004 Add prices
```

5 • Remote

Command	Use it when	Lesson
<code>git remote -v / add / set-url</code>	checking / connecting / changing the server	37, 38
<code>git fetch [--prune]</code>	seeing what changed, safely	40
<code>git pull [--rebase / --ff-only]</code>	updating your branch	41
<code>git push [-u origin BRANCH]</code>	publishing	42, 43
<code>git push --force-with-lease</code>	replacing your own rewritten branch	42
<code>gh pr create / review / merge</code>	pull requests	52–54

Practise:

```
cd ~/git-practice/ref-remote
(cd grace && echo "chai" >> menu.txt && git commit -q -am "Add chai" && git push -q)
cd ada && git fetch -q && git status | sed -n 2p
git log --oneline main..origin/main
```

```
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
a95a032 (origin/main, origin/HEAD) Add chai
```

6 • Recovery

Command	Use it when	Lesson
<code>git reset --soft/--mixed/--hard REV</code>	moving your branch back (local commits)	31
<code>git revert COMMIT / -m 1 MERGE</code>	undoing pushed commits safely	32
<code>git reflog / ORIG_HEAD</code>	finding where you were	33, 79–81
<code>git fsck --lost-found / --unreachable</code>	last resort: dangling commits and blobs	79, 81
<code>git branch NAME SHA</code>	rescuing a found commit	79

Practise:

```
cd ~/git-practice/ref-recovery
git reset -q --hard HEAD~2
git reflog -2
git reset -q --hard ORIG_HEAD && git log --oneline -1
```

```
269869e (HEAD -> main) HEAD@{0}: reset: moving to HEAD~2
ecff18a HEAD@{1}: commit: Price mocha
ecff18a (HEAD -> main) Price mocha
```

7 • Advanced

Command	Use it when	Lesson
<code>git stash [push -m / pop / apply / list]</code>	parking unfinished work	34–36
<code>git cherry-pick [-x] COMMIT</code>	copying one commit (backports)	67
<code>git tag [-a/-s] vX.Y.Z / git describe</code>	releases	68, 69
<code>git bisect start / good / bad / run</code>	finding the commit that broke something	70
<code>git clean -n / -fd</code>	removing untracked files (preview first)	72
<code>git worktree add / list / remove</code>	two branches checked out at once	87
<code>git submodule add / update --init</code>	embedding another repository	86
<code>git lfs track / git lfs migrate</code>	large binary files	88

Practise:

```
cd ~/git-practice/ref-advanced
git bisect start HEAD "$(git rev-list --max-parents=0 HEAD)" > /dev/null
git bisect run bash check.sh 2>&1 | grep "is the first bad commit"
git bisect reset > /dev/null 2>&1
git stash list | wc -l
```

```
3ceea031d6f34d67cc8f33c993ea0784c54ae95f is the first bad commit
0
```

8 • Inside Git and security

Command	Use it when	Lesson
<code>git cat-file -t/-p OBJECT</code>	looking at blobs, trees, commits	74, 75
<code>git rev-parse REV / git symbolic-ref HEAD</code>	resolving names, HEAD	76, 77
<code>git hook run NAME, core.hooksPath</code>	testing and sharing hooks	82–85
<code>git log -S SECRET --all, git filter-repo</code>	finding and removing leaked data	89, 90
<code>git commit -S, git verify-commit, gpg.format ssh</code>	signed commits and tags	91

Practise:

```
cd ~/git-practice/ref-internals
git cat-file -t HEAD
git cat-file -p HEAD | head -2
git symbolic-ref HEAD
```

```
commit
tree edd9d5aca7be17de9c83a80dc687991f6f56e24d
parent fc345e6b28df7fd7d7f872b37d78b47d0d024103
refs/heads/main
```

Cleanup

```
cd ~ && rm -rf ~/git-practice/ref-repo ~/git-practice/ref-changes ~/git-practice/ref-branches ~/git-practice/ref-remote ~/git-practice/ref-recovery ~/git-practice/ref-advanced ~/git-practice/ref-internals
```

When a situation goes wrong rather than a command: [Module 22 · Troubleshooting](#).

Module 24 · Final capstone

Level 25 · ⌚ 4–6 hours · everything in the course · needs a GitHub account

The challenge

A team left you its repository in a bad state: meaningless commit messages, a password in the history, a large video in the history, no `.gitignore`, a version mismatch that fails CI, a feature branch that conflicts with `main`, a badly named tag, and no CI at all. Turn it into a **professional GitHub repository** with a complete delivery pipeline:

```
Developer → Git → GitHub → feature branch → Pull Request → review → checks → merge → tag  
→ release → GitHub Actions → Docker image → GHCR → Helm → Kubernetes
```

Start

```
bash 24-capstone/start-capstone.sh  
cp 24-capstone/check-capstone.sh ~/git-practice/
```

The repository is in `~/git-practice/capstone`. It is the cafe DevOps repository from [module 21](#): website, Dockerfile, Helm chart, scripts. Its CI and release workflows are in [21-devops-workflow/devops-project/.github/workflows](#).

Requirements

Clean, safe history (nothing is pushed yet, so you may rewrite everything)

1. Conventional Commit messages throughout; no `wip`, `asdf`, `fix`, `changes`.
2. No `.env`, no password, no file over 1 MB in **any** commit; `.gitignore` keeps them out from now on.
3. The image tag in `helm/cafe/values.yaml` follows `VERSION`; `scripts/ci.sh` passes on `main`.
4. No badly named tags; releases use annotated semantic version tags.

Professional GitHub repository

5. A public repository on your account; `main` protected: pull requests required, the CI check required and up to date, enforced for admins, no force pushes or deletions.
6. The workflows arrive through a pull request, with green checks and a review comment.
7. `feature/chai` arrives through a pull request: updated with `main`, its conflict resolved (latte at the agreed **3.40**), green checks, merged.
8. Documentation: README and `docs/CONTRIBUTING.md`.

Release and deployment

9. An annotated tag `v1.0.0` on `main` triggers the release workflow, which must succeed: the image built and pushed to GHCR, a GitHub release with the packaged Helm chart, and a deployment of that image with Helm to a Kubernetes (kind) cluster, verified by a smoke test.

Grade yourself

Inside your capstone repository, the local part:

```
bash ../check-capstone.sh
```

and, after publishing, the GitHub part too:

```
bash ../check-capstone.sh YOUR-ACCOUNT/git-course-capstone
```

capstone complete = done.

Hints

Requirement	Lessons
history rewriting	63 (interactive rebase), 66 (edit stops), 90 (<code>git filter-repo</code>)
secrets, large files	89, 90, 88, 73
conflicts	26, 27, 64
GitHub, protection, PRs	45–47, 52–54, 59
tags, releases, Actions	68, 69, 97, 98, module 21
Docker, Helm, Kubernetes	project 6

Reference

A complete, recorded run of the capstone (on the repository [git-course-capstone](#)) is in [walkthrough.md](#). Then take the [final exam](#).

```
rm -rf ~/git-practice/capstone ~/git-practice/check-capstone.sh
```

Glossary

The terms used in the course, in plain words, with the lesson that explains each one.

Term	Meaning	Lesson
annotated tag	a tag object with tagger, date, message (and optional signature); used for releases	69
bare repository	a repository without a working directory (only the <code>.git</code> contents); what servers store	37
bisect	binary search through history for the commit that introduced a bug	70
blame	for each line of a file, the commit that last changed it	71
blob	the object that stores a file's content (no name, no permissions)	74
branch	a movable name (reference) pointing to a commit; moves forward with each commit	17, 77
branch protection	server rules for a branch: required PRs, reviews, checks; no force pushes	59
checkout / switch	change the current branch (and the files)	19
cherry-pick	apply the change of one commit onto the current branch as a new commit	67
clone	copy a whole repository (all history) and set up <code>origin</code>	39
commit	a snapshot of the project plus author, date, message and parent commit(s)	11, 12
commit ID / hash / SHA	the 40-character hash naming a commit (shortened to 7 in most outputs)	12
conflict	the same lines changed differently on both sides of a merge or rebase; Git asks you	26
Conventional Commits	message format <code>type(scope): description</code> (feat, fix, docs ...), used for changelogs	84
credential helper	a program that stores and supplies HTTPS credentials (tokens) for Git	48
default branch	the branch a clone checks out and PRs target, usually <code>main</code>	45
deploy key	an SSH key registered on one repository, often read-only, for servers and CI	49
detached HEAD	<code>HEAD</code> points directly to a commit instead of a branch	78
diff	the line-by-line difference between two versions	15
fast-forward	a merge that only moves the branch label forward (no merge commit)	24
fetch	download new commits and update <code>origin/*</code> , without touching your branches	40
force-with-lease	a force push that refuses if someone else pushed in the meantime	42
fork	your own server-side copy of someone else's repository	38
gitignore	patterns of files Git should not track (<code>.gitignore</code>)	73
Git LFS	Large File Storage: big files stored outside Git, pointers inside	88
GitHub Actions	GitHub's CI/CD: workflows in <code>.github/workflows/</code> run on events such as push	98
HEAD	"where you are": normally points to the current branch	76
hook	a script Git runs automatically at an event (pre-commit, commit-msg, pre-push ...)	82
index / staging area	the list of what goes into the next commit	10
interactive rebase	<code>git rebase -i</code> : reorder, reword, edit, squash, fixup or drop commits	63
issue	a GitHub item tracking a bug, task or request; closed by <code>Fixes #N</code>	93
label	a tag on issues and PRs: type, priority, area	94

Term	Meaning	Lesson
merge	combine another branch into the current one	23
merge base	the most recent commit two branches share; the base of a three-way merge	25
merge commit	a commit with two (or more) parents	23
milestone	a group of issues/PRs with a goal and due date, often a release	95
object	blob, tree, commit or tag in <code>.git/objects</code> , named by the hash of its content	74
ORIG_HEAD	where HEAD was before the last reset, merge or rebase	31
origin	the default name of the remote you cloned from	37
origin/main	a remote-tracking branch: your clone's memory of the server's <code>main</code>	37
personal access token	a credential used instead of a password for Git over HTTPS and the API	48
pull	fetch + integrate (merge or rebase) into the current branch	41
pull request (PR)	a request to merge a branch, with diff, discussion, review and checks	51
push	upload commits and move the branch on the server (fast-forward only)	42
rebase	replay commits on top of another commit, creating new commits	60
reflog	local log of every position of HEAD and branches; the recovery tool	33
release	a GitHub page for a tag with notes and downloadable assets	97
remote	a named URL of another copy of the repository	37
reset	move the current branch to another commit (<code>--soft</code> , <code>--mixed</code> , <code>--hard</code>)	31
restore	discard working-directory changes or unstage (<code>--staged</code>)	29, 30
revert	a new commit that undoes an earlier one (safe for shared history)	32
semantic versioning	MAJOR.MINOR.PATCH: breaking / feature / fix	97
signed commit	a commit with a GPG or SSH signature proving who made it	91
squash	combine several commits into one (interactive rebase, or squash merge)	54, 63
stash	a shelf for uncommitted changes (<code>git stash</code> , <code>pop</code> , <code>apply</code>)	34
submodule	another repository embedded at a pinned commit	86
supply chain	everything your software is built from: dependencies, actions, images, their authors	92
tag	a fixed name for a commit, usually a version	68
three-way merge	a merge that compares both tips with their merge base	25
tree	the object for a folder: names, modes and the blob/tree IDs	74
upstream	the remote branch a local branch tracks (for push, pull, ahead/behind)	43
working directory	the files you see and edit	07
worktree	an additional working directory of the same repository, on another branch	87

Interview questions

Questions DevOps and platform engineers are asked about Git and GitHub, with the answers the course builds. Each answer names the lesson where you practised it; the best interview answers add a short "in my last project..." example.

Fundamentals

- 1. What is the difference between Git and GitHub?** Git is the version-control tool that runs on your computer and stores history in `.git`. GitHub is a hosting and collaboration service built around Git repositories: remotes, pull requests, reviews, Actions, releases. (Lessons 01–02, 44)
- 2. What are the working directory, the staging area and the repository?** The files you edit; the list of changes chosen for the next commit (`git add`); the committed history in `.git`. `git status` compares them. (Lessons 07–10)
- 3. What is a commit, exactly?** An object containing a tree (the snapshot), the parent commit ID(s), author, committer and message. Its ID is the hash of all of that, so changing anything creates a different commit. (Lessons 12, 74–75)
- 4. Why does a commit ID change after `--amend` even if the files are identical?** The committer date (and possibly message) changes, and the ID is a hash of the whole commit object. (Lesson 12)

Branching and merging

- 5. What is a branch in Git?** A reference: a small file containing one commit ID. Creating it is instant; committing moves it. (Lessons 17, 77)
- 6. Fast-forward vs three-way merge?** Fast-forward: the current branch has not moved, so its label moves forward, no merge commit. Three-way: both moved; Git compares both tips with their merge base and creates a merge commit with two parents. (Lessons 24–25)
- 7. How do you resolve a merge conflict?** Read `git status`, open the file, decide the correct content (talk to the other author if needed), remove all markers, `git add`, `git commit`; run the tests before pushing. `git merge --abort` if you need to step back. (Lessons 26–28)
- 8. Merge vs rebase: when do you use which?** Rebase your own unshared branch onto `main` to keep a linear history; merge (or squash-merge via PR) to integrate into shared branches. Never rebase commits others have built on. (Lessons 60–62)
- 9. What do squash, merge commit and rebase merge do on GitHub?** Squash: one new commit per PR. Merge commit: all commits plus a merge commit. Rebase merge: all commits replayed linearly, no merge commit. Each leaves a different history on `main`. (Lesson 54)

Undoing and recovery

- 10. `reset --soft`, `--mixed`, `--hard`?** All move the branch. Soft keeps changes staged; mixed (default) keeps them unstaged; hard discards them from the staging area and working directory. (Lesson 31)
- 11. `reset` vs `revert`?** Reset rewrites the branch (local, unpushed work). Revert adds a new commit undoing an old one: the only safe way on a shared branch. A merge is reverted with `-m 1`. (Lessons 31–32)
- 12. You ran `git reset --hard` by mistake. Can you recover?** Committed work: yes, with `git reflog` or `ORIG_HEAD`. Staged-only work: often, with `git fsck --lost-found` (dangling blobs). Never-staged edits: no. (Lessons 33, 81)

13. Someone deleted a branch on GitHub. How do you get it back? "Restore branch" on its last PR; or from any clone that still has `origin/NAME`; or your reflog if you worked on it; then push it back. (Lesson 79)

14. What is detached HEAD and is it dangerous? HEAD points at a commit, not a branch: normal when inspecting a tag or in CI. Commits made there belong to no branch; create a branch before leaving (`git switch -c`). (Lesson 78)

Remotes and collaboration

15. fetch vs pull ? Fetch downloads and updates `origin/*` only. Pull = fetch + merge or rebase into the current branch. Fetch first when you want to look before integrating. (Lessons 40–41)

16. Your push is rejected with "fetch first". What happened and what do you do? Someone pushed to the branch; the server's branch has commits you lack. Pull (preferably `--rebase`), resolve if needed, push again. Not `--force`. (Lesson 42, troubleshooting 8)

17. When is `--force-with-lease` appropriate? After rewriting your own feature branch (rebase, amend); it refuses if someone else pushed meanwhile, unlike `--force`. Never on `main`. (Lessons 42, 60)

18. What is an upstream branch? The remote branch a local branch tracks; it enables plain `git push / git pull` and ahead/behind counts. Set with `git push -u`. (Lesson 43)

19. Describe GitHub Flow and Git Flow. Which do you prefer? GitHub Flow: `main` always deployable, short branches, PRs, deploy on merge, ideal for continuously deployed services. Git Flow: `develop`, release and hotfix branches, useful for versioned products maintained in parallel; often too heavy for web services. (Lessons 57–58)

20. How do you enforce code review and CI? Branch protection on `main`: PRs required, approvals (often via CODEOWNERS), required status checks up to date, enforced for admins, no force pushes. (Lesson 59)

Advanced Git

21. How would you find which commit introduced a bug? `git bisect start BAD GOOD`, then `git bisect run ./test.sh` (exit 0 good, 1 bad, 125 skip). (Lesson 70)

22. How do you get a fix from `main` into a release branch? `git cherry-pick -x SHA` on the release branch, tag a patch release. Cherry-pick dependencies first if the fix builds on them. (Lessons 67, project 5)

23. Lightweight vs annotated tags? Lightweight: just a name. Annotated: an object with tagger, date, message, optional signature; required by `git describe` and best for releases. (Lessons 68–69)

24. What are Git hooks and how do teams share them? Scripts run at events (pre-commit, commit-msg, pre-push; pre-receive on servers). They are not cloned: teams commit them and set `core.hooksPath`, or use the pre-commit framework, and enforce the same checks in CI. (Lessons 82–85)

25. Submodules, worktrees, LFS: one sentence each. A submodule embeds another repository pinned to a commit; a worktree is another working directory of the same repository on another branch; LFS stores large binaries outside Git behind pointer files. (Lessons 86–88)

Internals

26. Name Git's object types. Blob (content), tree (folder), commit (snapshot + history), tag (annotated tag). All named by the hash of their content. (Lessons 74–75)

27. What does `git add` actually do? Writes the file's content as a blob into the object store and records it in the index; `git commit` then writes a tree from the index, a commit object, and moves the branch. (Lesson 75)

Security

28. A secret was committed and pushed. What do you do? Revoke or rotate it immediately (that is the fix); check access logs; then remove it from history with `git filter-repo` on a mirror clone, force-push, have everyone re-clone, ask GitHub to purge caches; prevent with `.gitignore`, scanners, push protection. (Lessons 89–90, troubleshooting 12)

29. Why does `git rm` not remove a secret? It only removes the file from the next snapshot; every earlier commit still contains it, in every clone. (Lesson 89)

30. Why sign commits, and how? Author fields are free text; a signature proves possession of a key. `gpg.format ssh`, `user.signingkey`, `commit.gpgsign true`, an allowed signers file for verification; GitHub shows Verified with the uploaded key. (Lesson 91)

31. What supply-chain risks exist around Git and GitHub Actions? Mutable tags and branches (pin actions by commit SHA), spoofed authors (require signatures), compromised accounts (2FA, least-privilege tokens), malicious commits in large changes (review, CODEOWNERS). (Lesson 92)

DevOps

32. Describe a release process you would build with Git. Conventional Commits, PRs into protected `main`, CI on every PR, an annotated semver tag triggers the release workflow: image to a registry, chart or artifacts to a release, deployment with Helm or GitOps; rollback = redeploy the previous tag. (Module 21, capstone)

33. How do you keep a long-running feature branch from becoming a merge nightmare? Merge or rebase `main` into it often, keep it short, split it into smaller PRs, use feature flags to merge unfinished work safely. (Lesson 56)

34. A deployment from tag `v1.2.0` fails because its image does not exist. How do you investigate and recover? `kubectl describe` / events show the pull error; the tag exists in Git but the image was never built. Roll back (`helm rollback`, or deploy the previous tag), then fix the pipeline so tags always build before deploying. (Project 6)

35. Why does CI fail with "no tag exactly matches" or an empty changelog? A shallow clone (`fetch-depth: 1`) has no tags or history; set `fetch-depth: 0` for release jobs. (Lessons 39, 69)